Construction and Statistical Analysis of Intensity–Duration–Frequency (IDF) Curves from 43 Years of High-Resolution NOAA Rainfall Data

Ojas Pathak · Roll No. 23B0428

CE343 | NOAA Station 01014000/01014007, Randolph, Alabama | Record Period: 1971–2013

Key Messages

1. The Log-Pearson Type III (LP3) distribution is a better fit than the Gumbel distribution for all six storm durations at this station, as confirmed by the Kolmogorov–Smirnov test. This is because the rainfall data is positively skewed — rare extreme events pull the upper tail of the data far to the right, something the Gumbel distribution cannot capture well.
2. The year 1971 contains an extreme storm event that the Grubbs test formally identifies as a statistical outlier in four out of six storm durations. Keeping this one event in the data inflates rare-event design intensities by up to 159% at the 15-minute duration compared to NOAA Atlas 14 reference values.
3. A split-period analysis (1971–1992 vs. 1993–2013) shows that extreme short-duration rainfall intensities were higher in the earlier half of the record, raising questions about whether the stationarity assumption — the fundamental assumption behind all classical IDF analysis — holds for this station.

Abstract

Intensity–Duration–Frequency (IDF) curves tell engineers how hard it can rain for a given storm duration and how often. They are the essential starting point for designing stormwater drains, culverts, and flood-protection systems. This report derives IDF curves for Randolph, Alabama (USA), using 43 years of 15-minute rainfall records from the NOAA DSI-3260 database. Every single calculation step — data parsing, statistical fitting, goodness-of-fit testing, outlier detection, and uncertainty quantification — was coded from scratch in Python without using any statistical solver library. Annual maximum rainfalls for six storm durations (15 minutes to 24 hours) were fitted to two distributions: Gumbel and Log-Pearson Type III (LP3). The Kolmogorov–Smirnov test selected LP3 as the better fit for all durations. The resulting IDF values were summarised by the empirical formula $i = 36.61 \cdot T^{0.271}/d^{0.742}$. Results were validated against NOAA Atlas 14 official estimates: agreement is excellent for common storms but diverges sharply for rare events, primarily due to a single extreme storm in 1971 that was confirmed as an outlier by the Grubbs test. Bootstrap resampling with 1000 iterations quantified the sampling uncertainty, which is very large for rare events at short durations.

1 Introduction

1.1 What Are IDF Curves and Why Do We Need Them?

Imagine you are designing a stormwater drain for a road or a building. One of the first questions you must answer is: how much water will this drain need to carry during a storm? To answer that, you need to know how intense the rainfall could get, over how long, and how rare that event would be. That is exactly what IDF curves tell you.

An IDF curve is a graph (or a table) with three pieces of information:

- **Intensity (i):** How fast rain falls, measured in mm/hr.
- **Duration (d):** How long the storm lasts — from 15 minutes for a quick cloudburst to 24 hours for a slow-moving weather system.
- **Frequency (Return Period T):** A "100-year storm" does not mean it only happens once every 100 years; it means there is a 1% chance of it happening in *any* given year.

The key engineering insight is this: a short, intense rainstorm (say 15 minutes) will have a much higher intensity than a long, gentle storm (say 24 hours), even if both have the same return period. IDF curves capture exactly this relationship.

1.2 Why Build From Scratch?

Most engineers use pre-made IDF curves from national atlases. In the United States, NOAA Atlas 14 [1] is the official reference. However, understanding *how* those curves are made is crucial to using them intelligently. This project builds the entire IDF derivation pipeline from raw data, implementing every statistical method by hand so that every assumption and every calculation is transparent and understandable.

1.3 Study Area and Data

The station used is NOAA station 01014000/01014007, located in Randolph, Alabama (latitude 32.89°N, longitude 86.88°W). Two station codes refer to the same physical gauge: the code changed in 1993 due to administrative reclassification. The dataset is the NOAA DSI-3260 15-Minute Precipitation Dataset, which contains the depth of rain that fell every 15 minutes, recorded in hundredths of

an inch. The record spans 1971 to 2013, giving 43 years of data.

Alabama sits in a subtropical humid climate with two main types of storms: warm season convective thunderstorms (short, very intense) and winter frontal systems (long, moderate intensity). This dual storm character is reflected in the shape of the IDF curves, as we will see.

1.4 Structure of This Report

This report walks through the entire analysis step by step, in the same order as the Python pipeline. Each section explains what we did, why we did it, how the maths works, what the code does, and what the results mean.

2 Data: Where It Comes From and How We Read It

2.1 The NOAA DSI-3260 Format

The raw data arrives as files with the extension `.dat`, compressed inside `.tar.Z` archives using an old Unix compression format called LZW (Lempel–Ziv–Welch). This is an older format than the common `.gz` format used today. To decompress it, we use the Python library `unlzw3`.

Each `.dat` file uses a *fixed-width* text format. This means every field occupies a specific, fixed number of characters in each line. There are no commas or tabs to separate the fields — you need to know exactly which character positions correspond to which piece of information. The format is called DSI-3260, which stands for "Data Set Identifier 3260". Each line starts with the record type code, then the station ID, then the element type code, then the year, month, and day, followed by a series of time/value/flag triplets — one for each 15-minute interval recorded that day.

2.2 Step 0: Decompressing the Archives

Before we can read a single raindrop measurement, we must extract the files from their `.tar.Z` containers. Step 0 of the pipeline does this.

Script: `step0_extract.py`
Input: `Dataset/state_data/01/*.tar.Z`
Output: Extracted `.dat` files in `Dataset/extracted/`

The logic is straightforward. The script reads each archive file as raw bytes, passes them to `unlzw3.unlzw()` which decompresses the LZW encoding and returns the bytes of a standard TAR archive. Python's built-in `tarfile` module then extracts the contents. The temporary `.tar` file is deleted afterwards to save disk space.

2.3 Step 1: Parsing the Raw Records

Once the `.dat` files are extracted, Step 1 reads every line and pulls out the actual rainfall measurements.

Script: `step1_parse.py`
Input: Extracted `.dat` files + `by_month` folders
Output: `output/clean_data.csv`

The parser filters only lines where:

1. The record type is `15M` (15-minute precipitation record).
2. The element type is `QPCP` (quantitative precipitation).
3. The station ID matches `01014000` or `01014007` (our station).

Each qualifying line contains up to 24 time/value/flag triplets. The value field is an integer in units of 0.01 inches. Values of `099999` indicate missing data, and flag characters `[`, `]`, `{` indicate accumulated or deleted records — all of these are skipped. Valid values are converted from hundredths of an inch to millimetres by multiplying by $0.01 \times 25.4 = 0.254$. The daily total record (time code `2500`) is also excluded since it would double-count rainfall. The 2011–2013 data arrives in a separate by-month folder structure, which is also scanned.

The output `clean_data.csv` contains columns: year, month, day, hour, minute, and `precip_mm`. After deduplication and sorting, this file contains tens of thousands of 15-minute measurements spanning 1971–2013.

A critical quality control note: The gauge records rainfall in 0.01-inch steps, which equals 0.254 mm. This quantisation error is relatively small for long storms but can be significant for short-duration maxima where the actual depth is only a few millimetres.

3 Building the Annual Maximum Series

3.1 What Is the Annual Maximum Series?

The Annual Maximum Series (AMS) is the backbone of this entire analysis. The idea is simple: from each year's rainfall record, we extract just one number per storm duration — the *highest* intensity recorded that year. After 43 years, we have 43 such annual maxima for each duration. These are the values we fit our statistical distributions to.

Why only the maximum? Because IDF curves are about extreme events. We do not care about average rainy days. We care about the worst storms. Using the annual maximum ensures we are focused on extremes.

Script: `step2_ams.py`
Input: `output/clean_data.csv`
Output: `output/ams.csv`

3.2 How the Rolling Window Works

For a 15-minute storm duration, the annual maximum is simply the highest 15-minute rainfall depth recorded in the year, divided by 0.25 hours, to give mm/hr. Easy.

For longer durations, the process is a bit more clever. We use a *rolling window*: we slide a window of the appropriate width across the entire year's 15-minute data and sum up the depths within it. For a 1-hour duration, the window is 4 steps wide (4×15 min). For a 24-hour duration, it is 96 steps wide. The maximum value of this rolling sum in any given year — converted to mm/hr by dividing by the duration in hours — is the annual maximum for that year and duration.

This approach correctly handles storms that span midnight (which a simpler calendar-block approach would split across two days) and captures the true peak accumulation regardless of when it starts.

The code first builds a complete 15-minute time series by resampling the cleaned data to a regular 15-minute grid. Missing 15-minute intervals (dry periods or data gaps) are filled with zero, since no rainfall was recorded. Then for each duration, `pandas.Series.rolling(window).sum()` computes the rolling accumulated depth, and `groupby(year).max()` extracts the annual peak.

3.3 The Resulting AMS Dataset

Table 1 shows summary statistics of the AMS for all six durations. Several patterns are immediately visible. First, the mean intensity drops dramatically with duration: the average annual maximum is about 97 mm/hr at 15 min but only 4.3 mm/hr at 24 hr. This is simply because the same storm deposits the same total depth; spread over a longer window, the intensity (depth per hour) is lower.

Second, the standard deviation is large relative to the mean, especially at short durations. The coefficient of variation (standard deviation divided by mean) at 15 min is approximately 0.70, meaning the data is highly variable from year to year. This variability is driven by the 1971 event, which we will examine in detail later.

Third, the maximum values in the 15-minute and 30-minute columns (629.9 and 315.0 mm/hr respectively, both from 1971) are dramatically larger than the next-highest values. These are the outlier values that will cause significant problems in our statistical analysis.

4 Fitting Statistical Distributions

4.1 Why Fit a Distribution at All?

We have 43 annual maxima. If we want to estimate what intensity might occur once every 100 years, we have a problem: we only have 43 data points, so we cannot directly observe a 100-year event from our record. Instead, we fit a mathematical distribution to the 43 data points and then use that distribution to predict beyond the range of the data. This is called *extrapolation*, and it is inherently uncertain — but it is the best we can do with a finite record.

Two distributions are fitted in this analysis: the Gumbel distribution and the Log-Pearson Type III distribution.

Script: `step3_distributions.py`
Input: `output/ams.csv`
Output: `output/params.csv`

4.2 The Gumbel Distribution (Extreme Value Type I)

The Gumbel distribution was developed specifically to model the maximum values of random variables — things like annual peak floods, annual maximum temperatures, and annual maximum rainfalls. It is also called the Extreme Value Type I (EV-I) distribution.

The mathematical form of the Gumbel cumulative distribution function (CDF) is:

$$F(x) = \exp\left(-e^{-(x-u)/\alpha}\right) \tag{1}$$

Here, $F(x)$ is the probability that the annual maximum intensity is less than or equal to x. The two parameters are:

- u = location parameter (roughly, the most likely value)
- α = scale parameter (how spread out the distribution is)

We estimate these parameters from the data using the **Method of Moments**: we match the mean and standard deviation of the theoretical distribution to those computed from our 43 data points. The formulas are:

$$\alpha = \frac{\sqrt{6}}{\pi} \cdot s \tag{2}$$

$$u = \bar{x} - 0.5772 \cdot \alpha \tag{3}$$

where $\bar{x}$ is the sample mean and s is the sample standard deviation (with Bessel's correction, i.e., dividing by $n-1$). The constant 0.5772 is the Euler–Mascheroni constant γ.

Once we have the parameters, the intensity corresponding to a return period T (the "T-year storm") is found by inverting Equation 1:

$$x_T = u - \alpha \ln\left(-\ln(1 - 1/T)\right) \tag{4}$$

For example, a 100-year event has a non-exceedance probability of $1 - 1/100 = 0.99$. Plugging in: $-\ln(-\ln(0.99)) \approx 4.60$, so $x_{100} = u + 4.60\,\alpha$.

4.3 The Log-Pearson Type III Distribution

The Log-Pearson Type III (LP3) distribution is the standard distribution recommended by the US Federal Interagency Committee on Water Data (Bulletin 17C) for flood frequency analysis in the United States. It is also widely used for rainfall frequency analysis.

The LP3 is not fitted to the raw intensity values x. Instead, it is fitted to the **base-10 logarithms** of the intensity values: $y = \log_{10}(x)$. In the log-space, the data follows a Pearson Type III distribution, which is a generalised gamma distribution that can handle both positive and negative skewness.

The three parameters are estimated by Method of Moments applied to the log-transformed data:

$$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i \tag{5}$$

$$s_y = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{6}$$

$$g = \frac{n\sum_{i=1}^{n}(y_i - \bar{y})^3}{(n-1)(n-2)\,s_y^3} \tag{7}$$

where $\bar{y}$ is the mean of the log-transformed values, s_y is their standard deviation, and g is the skewness coefficient. The skewness g is crucial: it tells us how asymmetric the tail of the distribution is. If $g > 0$, the right tail is heavy (rare events are very large); if $g < 0$, the left tail is heavy (very small events are rarer than you would expect from a symmetric distribution); if $g = 0$, it reduces to the normal distribution (in log-space), which is the log-normal distribution.

Table 1: Summary statistics of the Annual Maximum Series (AMS) for all six storm durations. All intensities in mm/hr. $n = 43$ years (1971–2013). The 1971 event dominates the maximum values at short durations.

Statistic	15 min	30 min	1 hr	2 hr	6 hr	24 hr
Mean (mm/hr)	97.06	71.64	46.72	28.79	13.12	4.68
Std dev	88.70	41.59	18.67	9.92	4.01	1.39
Min	50.80	40.64	22.86	11.43	6.77	2.12
Median (50%)	91.44	71.12	43.18	27.94	12.70	4.44
75th percentile	101.60	81.28	53.34	33.02	14.82	5.61
Maximum	**629.92**	**314.96**	**157.48**	78.74	26.25	8.36
Year of max	**1971**	**1971**	**1971**	1971	1979	2004

4.3.1 The Wilson–Hilferty Frequency Factor

To find the T-year quantile from LP3, we need the *frequency factor* K_T. The exact calculation requires integrating the Pearson Type III density, which has no closed form. Instead, we use the **Wilson–Hilferty (1931)** cube-root approximation:

$$K_T = \frac{2}{g}\left[\left(1 + \frac{g\,z}{6} - \frac{g^2}{36}\right)^3 - 1\right] \tag{8}$$

where z is the standard normal quantile corresponding to the non-exceedance probability $p = 1 - 1/T$. This approximation is remarkably accurate over the range of g and T values encountered in hydrology.

4.3.2 Inverse Standard Normal CDF

Since no statistical library is used, the standard normal inverse CDF (needed to find z from p) is computed using a rational polynomial approximation from Abramowitz and Stegun (1964). This approximation is accurate to about 4 decimal places, which is more than sufficient for engineering purposes.

4.3.3 Back-transforming to Intensity

Once K_T is computed, the T-year log-intensity in log-space is:

$$\log_{10}(x_T) = \bar{y} + K_T \cdot s_y \tag{9}$$

and the actual intensity is recovered by raising 10 to that power:

$$x_T = 10^{\bar{y} + K_T \cdot s_y} \tag{10}$$

4.4 Fitted Parameters

Table 2 shows the fitted parameters for both distributions at all six durations. Several important trends emerge:

- Both distributions show decreasing location/mean parameters as duration increases, reflecting the lower intensities at longer durations.
- The LP3 skewness g decreases systematically from 1.90 (15 min) to -0.28 (24 hr). Positive skewness at short durations indicates that rare events are disproportionately intense (the 1971 storm is a good example). The slightly negative skewness at 24 hr means the 24-hour maximum series is more symmetric.
- The Gumbel scale parameter α is very large at 15 min (68.4) relative to the location u (75.6), meaning the distribution is wide and the tail is heavy. This is physically consistent with the high variability of short-duration convective storms.

5 Choosing the Better Distribution: The KS Test

5.1 The Problem of Model Selection

We have fitted two different distributions to our data. They will give different estimates of the 100-year storm. Which one should we use? We need an objective test to answer this question.

Script: `step4_ks_test.py`
Input: `output/ams.csv`, `output/params.csv`
Output: `output/ks_results.csv`

5.2 What the KS Test Does

The Kolmogorov–Smirnov (KS) test is a non-parametric goodness-of-fit test. "Non-parametric" means it makes no assumptions about what distribution the data follows — it just directly compares the shape of the data to the shape of the candidate distribution.

The idea is intuitive. We have 43 data points. Sort them from smallest to largest. The *empirical* CDF (ECDF) is the staircase function that jumps by $1/n$ at each data point — it represents what the data tells us about the cumulative probability. The *theoretical* CDF is the smooth curve predicted by the fitted distribution (Gumbel or LP3). The KS test asks: how far apart are these two curves?

The empirical CDF at data point x_i (ranked i-th out of n) is estimated using the **Weibull plotting position**:

$$\hat{F}(x_i) = \frac{i}{n+1} \tag{11}$$

This formula has the property that it avoids the impossible values of 0 and 1 at the extremes.

The KS statistic D_n is the maximum absolute difference between the empirical and theoretical CDFs across all data points:

$$D_n = \max_{i=1,\ldots,n} \left|\hat{F}(x_i) - F_{\text{theory}}(x_i)\right| \tag{12}$$

A small D_n means the theoretical distribution closely hugs the data. A large D_n means there is a big mismatch somewhere.

5.3 The Critical Value and the Decision Rule

We compare D_n against a critical value D_{crit}. If $D_n D_{\text{crit}}$, the test *passes*: we cannot reject the hypothesis that the data comes from this distribution. If $D_n > D_{\text{crit}}$, the test *fails*: the distribution is a poor fit.

Table 2: Fitted distribution parameters for the Annual Maximum Series at NOAA Station 01014000/01014007, Randolph, Alabama ($n = 43$ years). Gumbel parameters are in mm/hr. LP3 parameters are in the $\log_{10}$ domain (dimensionless $\bar{y}$ and s_y; g is the skewness coefficient).

Duration	u (Gumbel)	α (Gumbel)	$\bar{y}$ (LP3)	s_y (LP3)	g (LP3)
15 min	75.574	68.424	2.0081	0.1854	**1.8958**
30 min	59.678	32.308	1.8620	0.1511	1.5992
1 hr	38.124	16.434	1.6503	0.1454	1.1072
2 hr	24.018	8.945	1.4377	0.1534	0.2401
6 hr	11.351	3.582	1.1048	0.1413	0.2173
24 hr	4.173	1.256	0.6659	0.1490	**−0.2831**

At significance level $\alpha = 0.05$ (the 5% significance level, meaning we accept a 5% probability of wrongly rejecting a good distribution), the critical value for sample size n is approximated as:

$$D_{\text{crit}} = \frac{1.36}{\sqrt{n}} \tag{13}$$

For $n = 43$: $D_{\text{crit}} = 1.36/\sqrt{43} = 1.36/6.557 = 0.2074$.

After both distributions are tested, the one with the smaller D_n (and that also passes the test) is selected as the best fit.

5.4 Computing the LP3 CDF

For the Gumbel distribution, evaluating $F(x)$ at any point is trivial using Equation 1. For LP3, there is no simple closed-form CDF. The code builds a fine probability grid from $p = 0.0001$ to 0.9999 with 50,000 points, computes the corresponding LP3 quantile at each probability using the Wilson–Hilferty approximation, and then uses `numpy.interp()` to look up the probability for any given x value. This approach is fast, accurate, and avoids the need for numerical integration.

5.5 KS Test Results

Table 3 presents the full results. The critical value is $D_{\text{crit}} = 0.2074$ for all durations (since $n = 43$ for all).

The most important result is at the 15-minute duration: Gumbel **fails** the KS test ($D = 0.2277 > 0.2074$), while LP3 passes comfortably ($D = 0.1808$). This failure occurs because the Gumbel distribution cannot fit the heavy upper tail caused by the 1971 extreme event. The Gumbel has a fixed, moderate tail weight (its shape parameter is fixed at zero in the GEV framework). LP3, through its positive skewness parameter $g = 1.90$, bends its upper tail upward to accommodate the extreme observation.

At all other durations, both distributions pass, but LP3 always has a smaller D_n and is therefore selected as the better fit. Across the six durations, LP3's D_n ranges from 0.0664 to 0.1808, while Gumbel's ranges from 0.0666 to 0.2277.

5.6 Physical Interpretation

Why does LP3 consistently win? The answer lies in the skewness. Rainfall annual maxima in the southeastern United States are positively skewed because a small number of very intense convective events dominate the upper tail of the data. The Gumbel distribution is symmetric in its reduced variate and has a fixed, exponential upper tail. LP3, with its skewness parameter, can stretch its upper tail to match the data. At longer durations (6 hr, 24 hr), the positive skewness is smaller and the two distributions perform similarly — but LP3 still wins.

6 Computing IDF Intensities for Each Return Period

Script: `step5_return_periods.py`
Input: `output/params.csv`, `output/ks_results.csv`
Output: `output/idf_table.csv`

Using the LP3 parameters from Step 3 and the Wilson–Hilferty formula (Equations 8–10), we compute the rainfall intensity at six return periods $T \in \{2, 5, 10, 25, 50, 100\}$ years for each of the six storm durations. Table 4 is the primary engineering deliverable of this project: the full IDF table.

Reading the table across a row, we see how intensity grows as storms become rarer (larger T). Reading down a column, we see how intensity drops sharply as storm duration increases at any fixed return period. For example, at the 100-year return period, the 15-minute intensity (465.5 mm/hr) is nearly 49 times larger than the 24-hour intensity (9.6 mm/hr). Yet both represent the same rarity level. This is because the 15-minute peak comes from a concentrated convective burst, while the 24-hour event includes many hours of moderate rain.

The steepness of the growth from $T = 2$ to $T = 100$ is also strongly dependent on duration. The 15-minute intensity grows by a factor of 5.16 from 2 to 100 years, while the 24-hour intensity only grows by a factor of 2.04. This reflects the large positive skewness at short durations ($g = 1.90$): the LP3 tail is very heavy, so rare events are proportionately much more extreme.

7 The Empirical IDF Formula

Script: `step6_idf_formula.py`
Input: `output/idf_table.csv`
Output: `output/idf_formula_params.csv`

7.1 Why Fit a Formula?

The IDF table (Table 4) contains 36 individual intensity values (6 durations × 6 return periods). A design engineer might need intensities at, say, a 17-minute duration or a 15-year return period, which are not in the table. A fitted formula allows continuous interpolation between the

Table 3: Kolmogorov–Smirnov goodness-of-fit test results for Gumbel and LP3 distributions fitted to the AMS at NOAA Station 01014000/01014007. $D_{\text{crit}} = 0.2074$ at $\alpha = 0.05$ ($n = 43$). **Bold** = distribution fails the test. LP3 is selected as the best fit for all six durations.

Duration	D_{Gumbel}	Pass?	D_{LP3}	Pass?	Best fit
15 min	**0.2277**	**FAIL**	0.1808	PASS	LP3
30 min	0.1685	PASS	0.1598	PASS	LP3
1 hr	0.1626	PASS	0.1083	PASS	LP3
2 hr	0.1090	PASS	0.0918	PASS	LP3
6 hr	0.1069	PASS	0.1008	PASS	LP3
24 hr	0.0666	PASS	0.0664	PASS	LP3

Table 4: IDF table of design rainfall intensities (mm/hr) for NOAA Station 01014000/01014007, Randolph, Alabama. All values derived using the Log-Pearson Type III distribution with Method-of-Moments parameters fitted to 43 years of AMS data (1971–2013). The 2-year storm is sometimes called the "mean annual flood" event.

Duration	Return Period T (years)					
	2 yr	5 yr	10 yr	25 yr	50 yr	100 yr
15 min	90.2	132.6	176.7	258.9	346.5	465.5
30 min	66.8	91.8	115.0	153.7	191.1	237.4
1 hr	42.1	57.3	69.9	89.2	106.3	126.2
2 hr	27.0	36.7	43.4	52.3	59.2	66.3
6 hr	12.6	16.7	19.4	23.0	25.8	28.6
24 hr	4.7	6.2	7.1	8.2	8.9	9.6

tabulated values.

7.2 The Three-Parameter Power Law Formula

Following Koutsoyiannis et al. [2], the IDF relationship is parameterised as:

$$i(T, d) = \frac{a \cdot T^b}{d^e} \tag{14}$$

where:

- i = rainfall intensity (mm/hr)
- T = return period (years)
- d = storm duration (hours)
- a, b, e = fitted coefficients

The exponent b controls how rapidly intensity grows with return period. The exponent e controls how rapidly intensity decreases with duration. The coefficient a is a scaling factor.

7.3 Log-Linearisation and Least Squares

Taking the natural logarithm of both sides of Equation 14:

$$\ln(i) = \ln(a) + b\,\ln(T) - e\,\ln(d) \tag{15}$$

This is now a *linear* equation in the unknowns $[\ln a,\ b,\ e]$. For each of the 36 (T, d) data points, we write one such equation. Stacking all equations into matrix form:

$$\mathbf{y} = \mathbf{A}\,\boldsymbol{\beta} + \boldsymbol{\varepsilon} \tag{16}$$

where $\mathbf{y} = [\ln i_1, \ldots, \ln i_{36}]^\top$, $\boldsymbol{\beta} = [\ln a,\ b,\ e]^\top$, and $\mathbf{A}$ is the 36×3 design matrix with rows $[1,\ \ln T_j,\ -\ln d_j]$.

The ordinary least squares solution that minimises the sum of squared residuals in log-space is:

$$\hat{\boldsymbol{\beta}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{y} \tag{17}$$

This is solved numerically using `numpy.linalg.solve(`$\mathbf{A}^\top\mathbf{A}$`, `$\mathbf{A}^\top\mathbf{y}$`)`, which is more numerically stable than computing the matrix inverse explicitly.

7.4 Fitted Formula and Residuals

The fitted parameters are:

$$\boxed{i(T, d) = \frac{36.61 \cdot T^{0.2706}}{d^{0.7416}}} \tag{18}$$

with a root-mean-square error (RMSE) of 22.33 mm/hr across all 36 data points.

Interpreting the exponents:

- $b = 0.271$: Doubling the return period (e.g., from 10 to 20 years) increases the intensity by a factor of $2^{0.271} = 1.20$, or about 20%. This is a moderate return-period sensitivity.
- $e = 0.742$: Doubling the duration (e.g., from 1 hr to 2 hr) reduces the intensity by a factor of $2^{0.742} = 1.68$, or about 40%. This confirms that intensity decreases sub-linearly with duration — the rainfall total continues to accumulate as duration increases, just not fast enough to maintain the same intensity.

The RMSE of 22.33 mm/hr is acceptable but not negligible. Most of the error comes from the 15-minute, high-return-period entries, which the power-law formula slightly misrepresents because the LP3 tail at 15 min is so heavy that the simple power law in T cannot perfectly capture it.

8 Plotting the IDF Curves

Script: `step7_plot.py`
Input: `output/idf_table.csv`,

output/idf_formula_params.csv
Output: output/idf_curves.png

Figure 1 shows the final IDF curves. The dots are the 36 discrete intensities from the LP3 analysis (Table 4), and the solid lines are the continuous curves from the fitted formula (Equation 18). Both axes are logarithmic, which linearises the power-law relationship and makes the family of curves easier to read.

Key observations from Figure 1:

1. The curves are nearly parallel on a log–log plot, which is consistent with the power-law formula having the same exponents for all return periods. Small deviations from parallelism reflect the LP3 skewness effect — at short durations, the curves spread out more for higher return periods.
2. The steepest gradients (largest drop in intensity per unit increase in duration) occur at short durations. Moving from 15 min to 30 min roughly halves the intensity. Moving from 6 hr to 24 hr reduces it by a factor of only about 2.7.
3. The spacing between curves (which represents the effect of increasing the return period) is larger at short durations, confirming that rare events are proportionately much more extreme at short durations.
4. The fitted formula (solid lines) closely follows the LP3 points (dots) at all durations and return periods, with the largest deviations at 15 min for the highest return periods. This is because the formula is fitted simultaneously to all 36 points and compromises between them.

The plot was generated using `matplotlib` on a log–log scale (`ax.set_xscale('log')`, `ax.set_yscale('log')`). The x-axis ticks are set manually to the six standard durations in hours, with string labels converted back to minutes and hours for readability.

9 Validation Against NOAA Atlas 14

Script: step8_atlas14_compare.py
Input: output/idf_table.csv + hard-coded Atlas 14 values
Output: output/atlas14_comparison.csv, output/atlas14_comparison.png

9.1 What Is NOAA Atlas 14?

NOAA Atlas 14 [1] is the official reference for precipitation frequency estimates in the United States, published by the National Weather Service. Volume 9 covers the southeastern states. Atlas 14 is based on a *regional frequency analysis* using Partial Duration Series (PDS) and L-moments, pooling data from many stations across the region. It represents the best available official estimate.

Our analysis, by contrast, is based on a single station, the Annual Maximum Series, and the Method of Moments. The comparison therefore tells us how well a single-station analysis can reproduce what the much larger regional analysis produces.

9.2 Converting Atlas 14 Values

Atlas 14 publishes precipitation depths in inches. For example, the 1-hour, 10-year precipitation depth at our station is listed as 2.36 inches. We convert to intensity in mm/hr: $2.36 \times 25.4/1.0 = 59.94$ mm/hr.

9.3 Comparison Results

Table 5 shows the full comparison. The percentage difference is defined as:

$$\%\,\text{diff} = \frac{i_{\text{this study}} - i_{\text{Atlas14}}}{i_{\text{Atlas14}}} \times 100$$

A positive value means our estimate is higher than Atlas 14.

9.4 Discussion of Comparison Results

At the 2-year return period: Agreement is excellent across all durations. Our values are within ±4% of Atlas 14. This makes sense: the 2-year event is close to the sample median, which is robustly estimated from 43 years of data and does not depend much on the distribution's tail behaviour.

At the 5-year return period: Small positive biases emerge at short durations (+18.7% at 15 min, +12.9% at 30 min). These are modest and within the range of sampling variability.

At 10, 25, 50, and 100-year return periods: The bias grows progressively and dramatically at short durations. At $T = 100$ yr and 15 min duration, our estimate (465.5 mm/hr) is 159% higher than Atlas 14 (179.8 mm/hr). This is a factor-of-2.6 overestimation, which would lead to dramatically over-engineered (and over-expensive) drainage infrastructure if used directly for design.

At the 24-hour duration: The bias is much smaller and even reverses slightly at $T = 50$ and $T = 100$ yr (slight underestimation). This is because the 1971 storm is NOT an outlier at the 24-hour duration (as we will see in the Grubbs test section), so its influence at long durations is milder.

The convergence of Atlas 14 and our estimates at long durations, combined with the dramatic divergence at short durations and high return periods, points directly to the 1971 extreme event as the culprit. A single data point in 43 years is dominating the LP3 fit at short durations and pulling the upper tail to unrealistically high values.

10 Sensitivity Analysis: Gumbel vs. LP3

Script: step9_sensitivity.py
Input: output/params.csv, output/ks_results.csv
Output: output/sensitivity_comparison.csv, output/sensitivity_comparison.png

10.1 Why Compare Two Distributions Side by Side?

Even though the KS test selected LP3 as the best fit, it is instructive to see *how different* the Gumbel and LP3 estimates actually are. If they give almost the same answer, the choice of distribution does not matter much in practice.

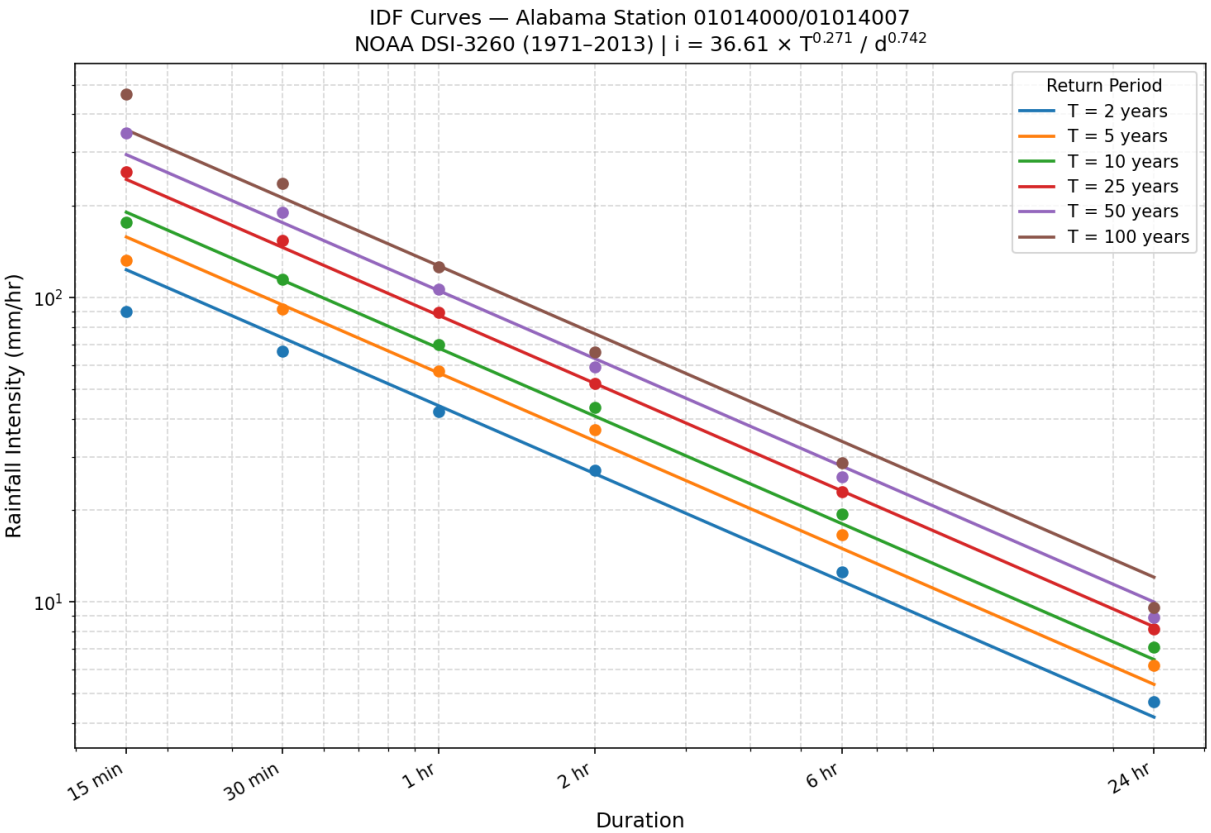

Figure 1: IDF curves for NOAA Station 01014000/01014007, Randolph, Alabama, derived from 43 years of 15-minute NOAA DSI-3260 data (1971–2013). Dots = LP3 design intensities from Table 4. Solid lines = fitted empirical formula $i = 36.61\,T^{0.271}/d^{0.742}$ (Equation 18). Both axes are log-scale. Each curve corresponds to a return period T, from 2 years (bottom) to 100 years (top). Duration labels on the x-axis correspond to the six standard design durations.

If they differ wildly, the choice is critical. Figure 3 and Table 6 show this comparison.

10.2 The Crossover Behaviour at 15 Minutes

The most interesting behaviour in Table 6 and Figure 3 is at the 15-minute duration. For return periods $T \leq 25$ yr, Gumbel gives *higher* estimates than LP3 (by up to 26%). But for $T = 100$ yr, LP3 gives a *higher* estimate than Gumbel (by 19%). The crossover happens somewhere around $T = 50$ yr.

Why does this happen? It is a consequence of the LP3's positive skewness ($g = 1.90$). A positively skewed distribution has:

- A lighter left tail (fewer very small annual maxima than Gumbel predicts)
- A lighter middle body (lower median and moderate-period estimates)
- A much heavier right tail (much larger extreme-period estimates)

So LP3 and Gumbel essentially "cross" in the upper tail. The 1971 event pushes the LP3 tail extremely far to the right, making it give enormous estimates at $T = 100$ yr while being more conservative at $T = 10$ or 25 yr.

At the 2-hour, 6-hour, and 24-hour durations, the skewness is much smaller, and LP3 consistently gives slightly lower estimates than Gumbel across all return periods. This is because with near-zero or slightly negative skewness, LP3 has a slightly lighter upper tail than Gumbel.

10.3 Engineering Implications

For a hydraulic engineer designing a small urban stormwater drain (which typically uses the 2-year to 25-year return period), the choice of distribution matters enormously at short durations: LP3 gives designs up to 26% smaller than Gumbel for moderate return periods. For large infrastructure with a 100-year design standard, LP3 gives designs up to 19% larger. These differences translate directly to pipe diameters, culvert sizes, and construction costs.

11 Grubbs Outlier Detection Test

Script: `step11_outlier_detection.py`
Input: `output/ams.csv`, `output/params.csv`
Output: `output/outlier_results.csv`, `output/idf_table_no_outlier.csv`, `output/outlier_analysis.png`

11.1 What Is the Grubbs Test?

The Grubbs test [3] is a classical statistical test designed to detect a single outlier in a dataset that is assumed to be normally distributed. An *outlier* is a data point that is so far from the rest of the data that it is unlikely to have come from the same underlying process.

In the context of rainfall data, an outlier does not neces-

Table 5: Comparison of derived LP3 IDF intensities (mm/hr) with NOAA Atlas 14 Volume 9 Version 2 reference values for NOAA Station 01014000/01014007 (Randolph, AL, lat. 32.89°N, lon. 86.88°W). Percentage differences greater than ±20% are highlighted in bold. Positive percentage = this study is higher than Atlas 14.

	$T = 2$ yr		$T = 5$ yr		$T = 10$ yr		$T = 25$ yr		$T = 50$ yr		$T = 100$ yr	
Dur.	Ours	%	Ours	%	Ours	%	Ours	%	Ours	%	Ours	%
15min	90.2	−2.6	132.6	+18.7	176.7	+39.2	258.9	+74.5	346.5	+111.9	**465.5**	**+158.9**
30min	66.8	−0.4	91.8	+12.9	115.0	+24.4	153.7	**+42.8**	191.1	**+45.8**	**237.4**	**+66.3**
1hr	42.1	−3.6	57.3	+8.9	69.9	+16.6	89.2	**+26.3**	106.3	**+35.1**	**126.2**	**+44.5**
2hr	27.0	+0.3	36.7	+13.8	43.4	+17.9	52.3	**+20.1**	59.2	**+21.1**	66.3	**+21.4**
6hr	12.6	+3.2	16.7	+13.8	19.4	+14.8	23.0	+12.7	25.8	+9.9	28.6	+7.0
24hr	4.7	+6.9	6.2	+14.3	7.1	+11.4	8.2	+4.1	8.9	−2.3	9.6	−8.3

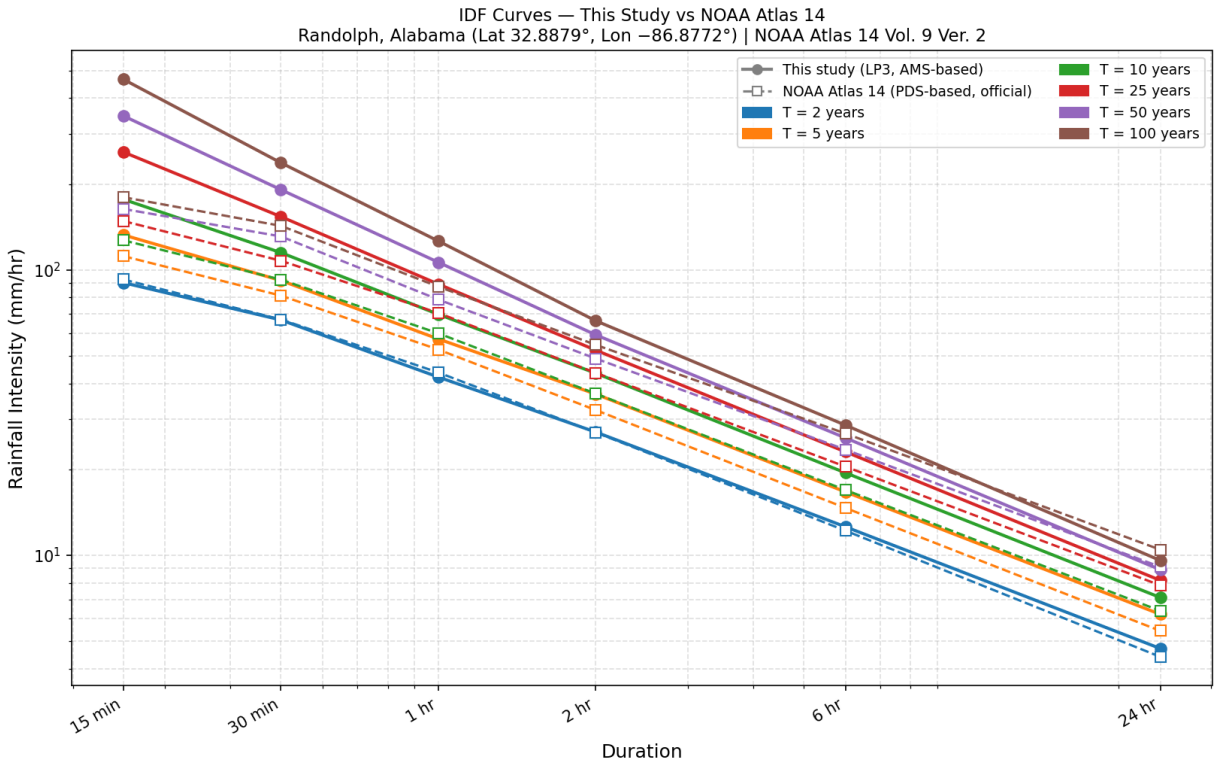

Figure 2: Comparison of IDF curves derived in this study (solid lines with filled circles) versus NOAA Atlas 14 Volume 9 reference values (dashed lines with open squares) for six return periods. Both axes are log-scale. Agreement is excellent at $T = 2$ years. Systematic overestimation develops at higher return periods, especially at short durations, driven primarily by the 1971 outlier event. Atlas 14 source: https://hdsc.nws.noaa.gov/pfds/ (retrieved March 2026).

sarily mean the event was erroneously measured. It could be a genuinely extreme but real storm that is simply much rarer than what our 43-year record can reliably characterise. Whether to include or exclude it is a hydrological judgement, but the Grubbs test gives us an objective, statistically defensible answer to whether the point is unusual by the standards of the data.

11.2 The Test Statistic

The Grubbs statistic is:

$$G = \frac{\max_i |x_i - \bar{x}|}{s} \tag{19}$$

where $\bar{x}$ is the sample mean and s is the sample standard deviation. This is simply the maximum absolute deviation from the mean, expressed in units of standard deviations. It identifies whichever data point is most extreme (either the largest or the smallest) as the suspect outlier.

11.3 The Critical Value

The critical value G_{crit} depends on the sample size n and the significance level α (we use $\alpha = 0.05$). It is derived from the t-distribution:

$$G_{\text{crit}} = \frac{n-1}{\sqrt{n}} \sqrt{\frac{t^2_{\alpha/(2n),\,n-2}}{n-2+t^2_{\alpha/(2n),\,n-2}}} \tag{20}$$

where $t_{\alpha/(2n),\,n-2}$ is the critical value of the t-distribution with $n-2$ degrees of freedom at the two-tailed level $\alpha/(2n)$. Since we have no statistical library, the t inverse is computed using the Cornish–Fisher expansion of the normal quantile, which works well for the degrees of freedom encountered here ($n - 2 = 41$).

For $n = 43$ and $\alpha = 0.05$: $G_{\text{crit}} = 3.0663$.

The rule is: if $G > G_{\text{crit}}$, the most extreme data point is an outlier at the 5% significance level.

Table 6: Sensitivity of design intensities (mm/hr) to the choice of distribution. The percentage difference is (LP3 − Gumbel) / Gumbel × 100. Positive values mean LP3 gives a higher estimate than Gumbel. A checkmark (✓) indicates the distribution passes the KS test; a cross (×) indicates it fails.

Duration	Distribution	$T = 2$	$T = 5$	$T = 10$	$T = 25$	$T = 50$	$T = 100$
15 min	Gumbel (×)	100.7	178.2	229.6	294.4	342.6	390.3
	LP3 (✓)	90.2	132.6	176.7	258.9	346.5	465.5
	% diff	−10.4	−25.6	−23.0	−12.1	+1.2	+19.3
30 min	Gumbel (✓)	71.5	108.1	132.4	163.0	185.7	208.3
	LP3 (✓)	66.8	91.8	115.0	153.7	191.1	237.4
	% diff	−6.6	−15.1	−13.1	−5.7	+2.9	+14.0
1 hr	Gumbel (✓)	44.1	62.8	75.1	90.7	102.3	113.7
	LP3 (✓)	42.1	57.3	69.9	89.2	106.3	126.2
	% diff	−4.6	−8.8	−7.0	−1.7	+4.0	+11.0
2 hr	Gumbel (✓)	27.3	39.6	47.9	58.2	66.0	73.7
	LP3 (✓)	27.0	36.7	43.4	52.3	59.2	66.3
	% diff	−1.0	−7.3	−9.4	−10.2	−10.3	−10.0
6 hr	Gumbel (✓)	12.6	17.9	21.5	26.1	29.6	33.0
	LP3 (✓)	12.6	16.7	19.4	23.0	25.8	28.6
	% diff	−0.1	−6.3	−9.7	−11.8	−12.8	−13.4
24 hr	Gumbel (✓)	4.8	6.5	7.6	9.0	10.1	11.1
	LP3 (✓)	4.7	6.2	7.1	8.2	8.9	9.6
	% diff	−1.1	−4.3	−6.3	−8.8	−11.5	−13.7

11.4 Grubbs Test Results

Table 7 presents the full Grubbs test results for all six durations.

Table 7: Grubbs outlier test results for the Annual Maximum Series at six storm durations. $G_{\text{crit}} = 3.0663$ for all durations ($n = 43$, $\alpha = 0.05$). The 1971 storm is a statistically significant outlier at the four shortest durations.

Duration	G	G_{crit}	Outlier?	Year	Value (mm/hr)
15 min	**5.867**	3.066	**YES**	1971	629.9
30 min	**5.711**	3.066	**YES**	1971	315.0
1 hr	**5.213**	3.066	**YES**	1971	157.5
2 hr	**4.320**	3.066	**YES**	1971	78.7
6 hr	2.792	3.066	no	—	—
24 hr	2.149	3.066	no	—	—

The 1971 event is confirmed as a statistically significant outlier at the 15-min, 30-min, 1-hr, and 2-hr durations, with Grubbs statistics ranging from 4.32 to 5.87 — all massively exceeding the critical value of 3.07. The G statistics decrease as duration increases, reflecting that the 1971 storm becomes relatively less extreme at longer accumulation periods. At the 6-hour duration, $G = 2.793.07$, meaning it is no longer a statistical outlier, and at 24 hours $G = 2.15$, clearly not extreme by that metric.

This pattern makes physical sense. The 1971 storm was an intense convective event — it produced an enormous amount of rain in a very short time. The peak intensity was extraordinarily high, but over a 24-hour window, the total accumulated depth was within the normal range for Alabama storms.

11.5 Impact of the Outlier on IDF Estimates

After the Grubbs test, the script refits LP3 to the data *without* the 1971 observation and recomputes the IDF table. The impact is dramatic at short durations and high return periods. Table 8 shows the percentage change in IDF intensity when the outlier is removed.

At the 15-minute duration, the 100-year intensity drops from 465.5 to 187.6 mm/hr — a 59.7% reduction. The no-outlier estimate (187.6 mm/hr) is in very close agreement with Atlas 14 (179.8 mm/hr), a difference of only +4.3%. This confirms that the Atlas 14 divergence we observed in Section 8 is almost entirely due to the 1971 event. Interestingly, at the 2-year return period, the no-outlier estimates are slightly *higher* than the with-outlier estimates (e.g., +12.8% at 2 hr). This is because removing the outlier changes the mean, standard deviation, and skewness of the LP3 fit simultaneously, and the effect on lower quantiles can be counterintuitive.

At 6-hour and 24-hour durations, the impact of removal is small and mostly positive (slightly higher estimates), because the 1971 event is not an outlier at these durations. When it was included, it was a "moderately extreme" point that added to the positive skewness; removing it reduces the skewness and slightly reduces the upper tail, but the effect is minor.

12 Temporal Trend Analysis: Is Rainfall Changing Over Time?

Scripts: `step10_trend_analysis.py`, `step10b_trend_no_outlier.py`
Input: `output/ams.csv`
Output: `output/trend_comparison.csv`, `output/trend_comparison_clean.csv`,

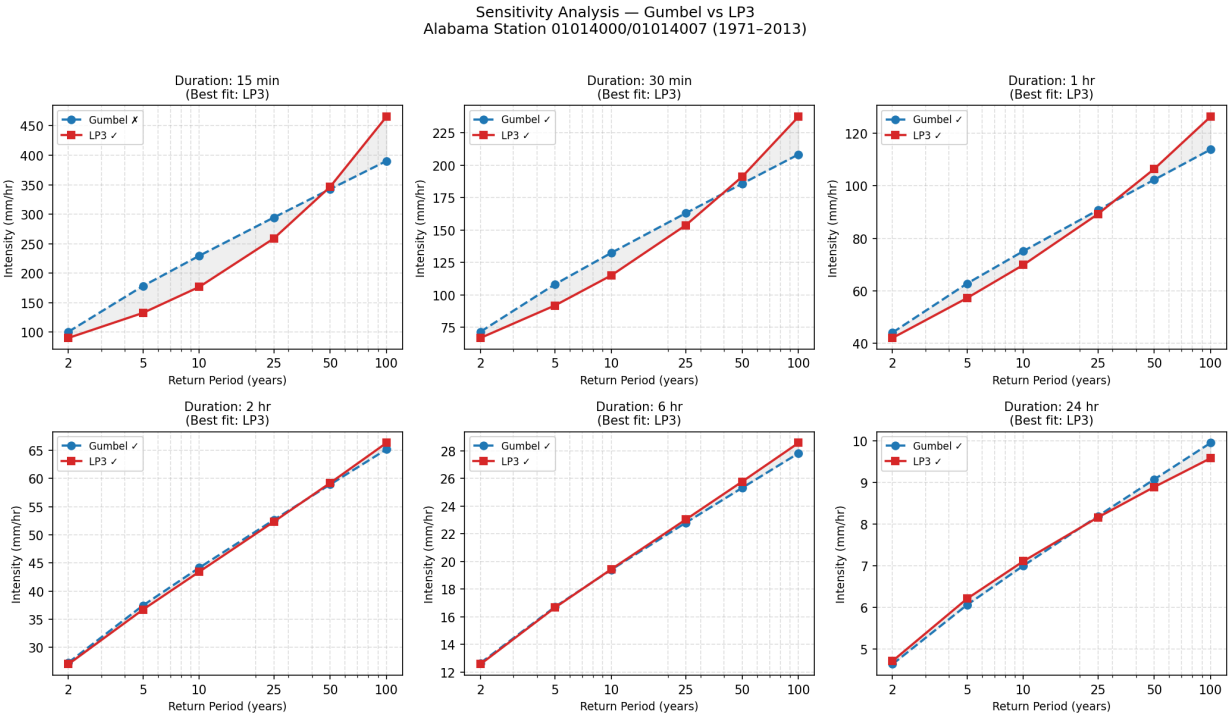

Figure 3: Sensitivity analysis comparing Gumbel (blue dashed) and LP3 (red solid) IDF estimates for all six storm durations. The grey shading represents the band of disagreement between the two distributions. Checkmark (✓) = passes KS test; cross (×) = fails KS test. At 15 min, the crossover behaviour between Gumbel and LP3 is clearly visible: LP3 gives lower estimates at moderate return periods but higher estimates at extreme return periods.

Table 8: Percentage change in LP3 IDF intensities when the 1971 outlier (629.9 mm/hr at 15 min; 314.96 mm/hr at 30 min; 157.48 mm/hr at 1 hr; 78.74 mm/hr at 2 hr) is removed from the AMS. Negative values mean the intensity estimate decreases when the outlier is excluded.

Duration	$T = 2$ yr	$T = 5$ yr	$T = 10$ yr	$T = 25$ yr	$T = 50$ yr	$T = 100$ yr
15 min	−1.0%	−9.6%	−22.8%	−39.3%	−50.2%	−59.7%
30 min	+2.1%	−7.8%	−19.1%	−33.4%	−43.0%	−51.7%
1 hr	+7.6%	−2.1%	−11.0%	−21.8%	−28.9%	−35.9%
2 hr	+12.8%	+6.9%	+0.4%	−8.2%	−14.5%	−20.5%
6 hr	+10.6%	+9.9%	+7.2%	+2.7%	−1.0%	−4.8%
24 hr	+7.4%	+8.1%	+7.2%	+5.4%	+3.8%	+2.1%

```
output/trend_analysis.png,
output/trend_analysis_clean.png
```

12.1 The Stationarity Assumption

Every classical IDF analysis assumes that the rainfall process is *stationary*: the statistical properties of annual maxima (mean, variance, skewness) are constant over time. A 100-year storm derived from a stationary model means the same thing in 1971 as in 2013 — the expected exceedance rate is always 1% per year.

If the climate is changing, or if land use around the station is changing (e.g., increasing urbanisation, which creates more heat-island convection), this assumption breaks down. Stationarity is now widely questioned in hydrology [8]. To test for it, we split the 43-year record into two approximately equal halves and fit separate LP3 distributions to each.

12.2 Split Period Setup

The record is split at year 1992:

- **Period 1 (1971–1992):** 22 years. Contains the 1971 outlier.
- **Period 2 (1993–2013):** 21 years. Post-1990s climate period.

LP3 is fitted independently to each sub-period's AMS using the same Method of Moments procedure, and IDF curves are computed for each sub-period. A second analysis removes the 1971 outlier before splitting.

12.3 Results With the Outlier

Table 9 shows selected results of the trend analysis with the outlier included. The difference between sub-periods is enormous at the 15-minute duration for high return periods: 691.5 vs. 187.6 mm/hr at $T = 100$ yr. However, this is almost entirely driven by the 1971 event being in Period 1. The period means and variances at short durations are heavily distorted by that single observation.

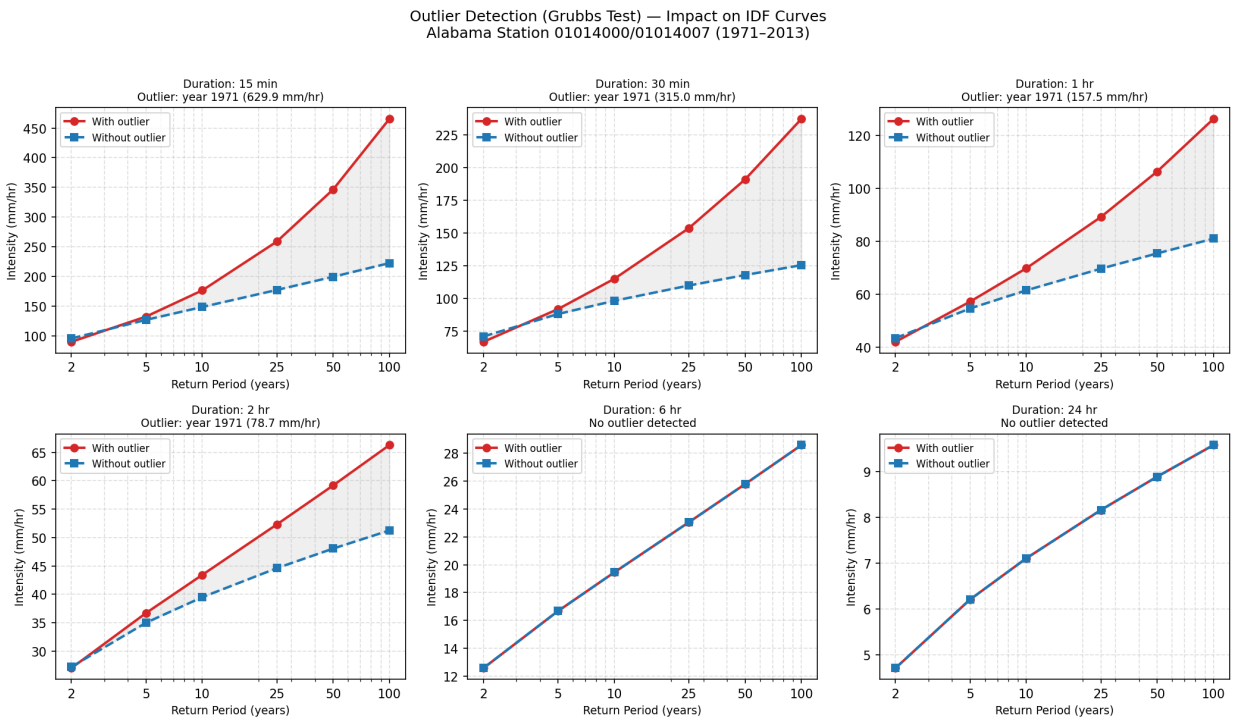

Figure 4: Impact of the 1971 outlier on IDF estimates. Each panel shows IDF curves computed with (red solid) and without (blue dashed) the outlier year. Grey shading = zone of influence of the outlier. At 15 min and 30 min, removing the outlier drastically reduces high-return-period estimates. At 6 hr and 24 hr, the outlier is not statistically significant and has little impact. The annotation shows the Grubbs test result for each duration.

12.4 Results Without the Outlier

The `step10b` script removes the 1971 event before splitting. The differences between sub-periods are substantially reduced but do not disappear entirely. At the 15-minute, 100-year event, the sub-period estimates are now about 300 mm/hr (1971–1992) vs. 188 mm/hr (1993–2013) — still a factor of 1.6 difference. This residual difference, which cannot be attributed to the outlier, is a genuine signal in the data.

At 6-hour and 24-hour durations, Period 2 consistently gives slightly higher intensities than Period 1 for all return periods. This is the opposite of the pattern at short durations. This could indicate a shift towards more prolonged, moderate-intensity synoptic rainfall events in the recent period, at the expense of short-duration convective extremes. This pattern is broadly consistent with reported changes in precipitation characteristics in the southeastern United States under warming climate conditions, though attribution is beyond the scope of this analysis.

12.5 The Stationarity Question

These results do not definitively prove non-stationarity — the sub-period samples (21–22 years each) are too small for robust statistical testing. However, they do raise a flag. If the 100-year short-duration intensity has genuinely declined from Period 1 to Period 2, then applying the full-record IDF curves may be overly conservative for future design. If the reverse is true (the 100-year 24-hour intensity has increased), the full-record curves may be non-conservative for long-duration design. A formal non-stationary frequency analysis would be needed to resolve this, using distributions with parameters that vary smoothly as a function of time.

13 Bootstrap Confidence Intervals

Script: `step12_bootstrap_ci.py`
Input: `output/ams.csv`
Output: `output/bootstrap_ci.csv`, `output/bootstrap_ci.png`

13.1 What Is the Bootstrap and Why Do We Need It?

Even if our LP3 distribution perfectly describes the true rainfall process, we have estimated its parameters from only 43 years of data. If we had a different 43-year record from the same station, the parameters would be slightly different, and the IDF estimates would be slightly different. The *sampling uncertainty* in our IDF estimates — the uncertainty arising from having a finite record — is quantified by the bootstrap confidence intervals.

The bootstrap [5] is a non-parametric resampling technique. The idea is beautifully simple: our 43-year sample *is* our best estimate of the underlying population. So we simulate "alternative realities" by drawing 43 values with replacement from our actual data, fitting LP3 to each such resample, and computing IDF estimates. After 1000 such iterations, we have 1000 bootstrap estimates of each IDF intensity. The 2.5th and 97.5th percentiles of these 1000 values form the 95% confidence interval.

13.2 Bootstrap Algorithm

The algorithm in `step12_bootstrap_ci.py`:

1. Set random seed to 42 for reproducibility.

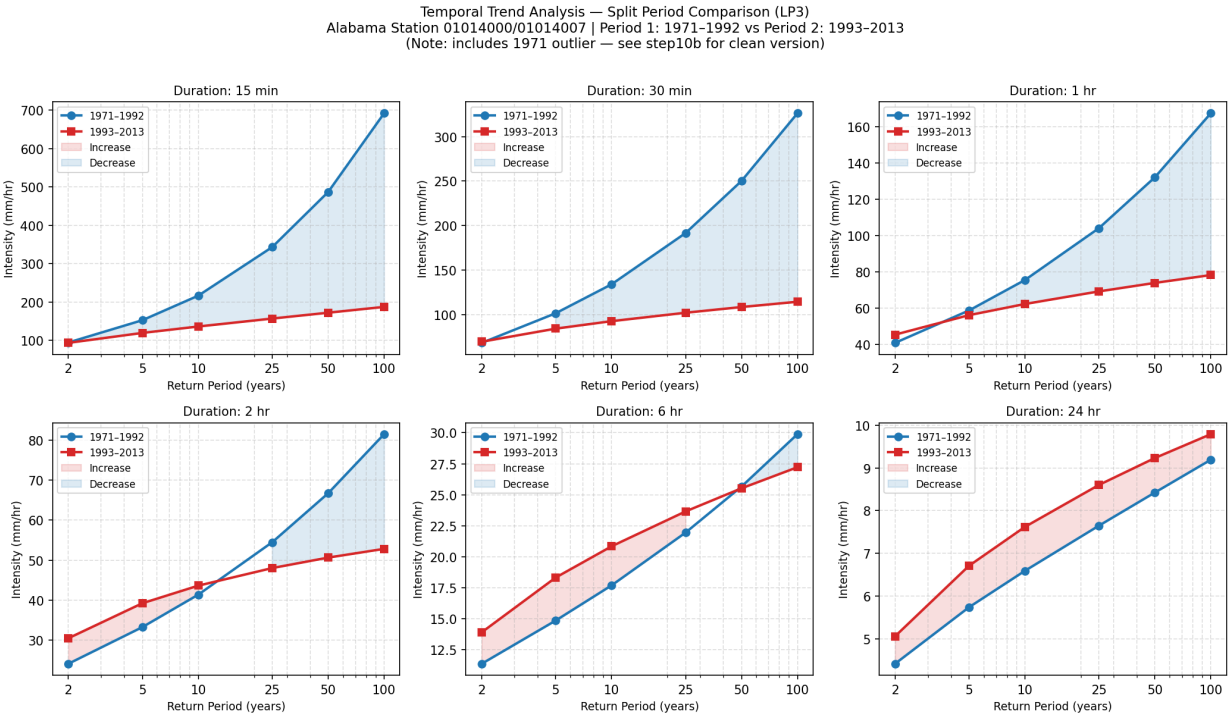

Figure 5: Split-period trend analysis. IDF curves for Period 1 (1971–1992, solid red) and Period 2 (1993–2013, dashed blue) for all six storm durations. The 1971 outlier is *included*. The enormous difference at 15 min and 30 min for high return periods is almost entirely attributable to the 1971 outlier. At longer durations, the periods are more comparable.

2. For each duration, extract the $n = 43$ AMS values.
3. Repeat 1000 times:
   (a) Draw 43 values with replacement using `numpy.random.randint`.
   (b) Fit LP3 parameters to the resample (Method of Moments).
   (c) Compute IDF intensities at all 6 return periods.
   (d) Store the 1000 estimates for each (T, d) pair.
4. For each (T, d) pair, compute the point estimate from the original data and the 2.5th/97.5th percentiles of the bootstrap distribution.

13.3 Bootstrap Results

Table 10 presents the 95% bootstrap confidence intervals.

13.4 Key Findings

The bootstrap results deliver one clear, sobering message: **rare-event estimates from 43 years of single-station data carry enormous uncertainty.**

At the 15-minute duration:

- The 2-year intensity has a narrow 95% CI of 83.7–105.0 mm/hr (a range of 21.3 mm/hr, or 24% of the point estimate). This is acceptable for engineering.
- The 100-year intensity has a 95% CI of 179.2–861.0 mm/hr (a range of 681.8 mm/hr, or 147% of the point estimate). The true 100-year intensity could be anywhere in this enormous range. Designing infrastructure based on the point estimate of 465.5 mm/hr is effectively a guess.

At the 24-hour duration, the uncertainty is much more manageable. The 100-year CI spans 8.1–10.9 mm/hr — a width of 2.8 mm/hr (29% of the estimate). This is because:

1. The 24-hour AMS has smaller variability (lower coefficient of variation).
2. The skewness is small ($g = -0.28$), so the LP3 tail is not extreme.
3. There is no single dominant outlier inflating the bootstrap distribution.

The practical implication is that for critical infrastructure designed to the 100-year standard, the uncertainty in the short-duration IDF estimates is so large that a regional multi-station analysis (like Atlas 14) is essential. Our single-station analysis is appropriate for understanding the local data and for designing to moderate return periods ($T \leq 25$ yr), but becomes unreliable for the most extreme events.

14 CDF Comparison Plots

Script: `step13_cdf_plot.py`
Input: `output/ams.csv`, `output/params.csv`
Output: `output/cdf_comparison.csv`, `output/cdf_comparison.png`

The CDF comparison plots give a visual, intuitive sense of how well each fitted distribution matches the empirical data. The script plots three things on the same graph for each duration:

1. **Empirical CDF (dots):** The Weibull plotting positions $\hat{F}(x_i) = i/(n+1)$ for the sorted AMS data. These are the actual data, plotted as cumulative probability.

Table 9: Selected split-period LP3 IDF intensities (mm/hr). The 1971 outlier is retained. Period 1: 1971–1992 (22 years). Period 2: 1993–2013 (21 years).

Duration	Period	$T = 2$	$T = 5$	$T = 10$	$T = 25$	$T = 50$	$T = 100$
15 min	1971–1992	94.9	152.9	216.8	343.2	486.6	691.5
	1993–2013	94.0	119.8	136.5	157.2	172.5	187.6
30 min	1971–1992	68.7	101.9	134.4	191.8	250.2	326.0
	1993–2013	70.0	84.6	93.0	102.5	108.8	114.8
1 hr	1971–1992	40.8	58.7	75.5	104.1	132.2	167.5
	1993–2013	45.3	56.1	62.2	69.2	73.9	78.2
2 hr	1971–1992	24.1	33.2	41.4	54.5	66.7	81.5
	1993–2013	30.5	39.2	43.6	48.0	50.6	52.8
6 hr	1971–1992	11.4	14.9	17.7	21.9	25.7	29.9
	1993–2013	13.9	18.3	20.8	23.7	25.5	27.2
24 hr	1971–1992	4.4	5.7	6.6	7.6	8.4	9.2
	1993–2013	5.1	6.7	7.6	8.6	9.2	9.8

Table 10: Bootstrap 95% confidence intervals for LP3 IDF intensities (mm/hr). All values in mm/hr. CI Width = upper bound − lower bound. Note how the CI width grows dramatically with return period, especially at short durations.

Duration	T (yr)	Point Est.	CI Lower (2.5%)	CI Upper (97.5%)	CI Width
15 min	2	90.2	83.7	105.0	21.3
	10	176.7	131.6	243.7	112.1
	25	258.9	152.4	400.7	248.3
	100	465.5	179.2	861.0	**681.8**
30 min	2	66.8	61.9	75.3	13.3
	10	115.0	91.2	146.7	55.5
	25	153.7	100.1	218.2	118.1
	100	237.4	112.0	400.4	288.4
1 hr	2	42.1	38.3	47.1	8.8
	10	69.9	56.8	87.2	30.4
	25	89.2	63.8	121.4	57.6
	100	126.2	72.1	198.6	126.5
24 hr	2	4.7	4.2	5.3	1.1
	10	7.1	6.3	7.8	1.5
	25	8.2	7.2	8.9	1.8
	100	9.6	8.1	10.9	2.8

1000 bootstrap resamples; random seed = 42.

2. **Gumbel CDF (blue line):** The smooth Gumbel CDF using the fitted parameters from Step 3.
3. **LP3 CDF (red line):** The LP3 CDF built by evaluating the Wilson–Hilferty formula on a fine probability grid and interpolating.

Looking at the 15-minute panel of Figure 7, the most striking feature is the rightmost data point at 629.9 mm/hr (1971). This point sits far to the right of where both fitted distributions would predict the highest observation to lie. The LP3 CDF is closer to it than Gumbel, but neither fits it comfortably — this is precisely the point the Grubbs test flagged as an outlier.

For durations of 1 hour and longer, both CDFs track the empirical data closely, with LP3 marginally better. The 24-hour panel shows nearly perfect agreement between LP3 and the empirical data — the small negative skewness ($g = -0.28$) is well captured, and the data cluster tightly around the fitted curve.

The CDF plot is also a useful diagnostic for checking for any systematic bias in the fit. If a fitted CDF consistently lies to the left of the empirical dots in the middle of the range, it means the distribution overestimates the probability of moderate events (i.e., underestimates the median intensity). No such systematic bias is visible for LP3 at any duration.

15 Summary of All Results

Table 11 provides a complete summary of every key numerical result from the full analysis pipeline, organised by finding. All values are discussed in detail in their respective sections above.

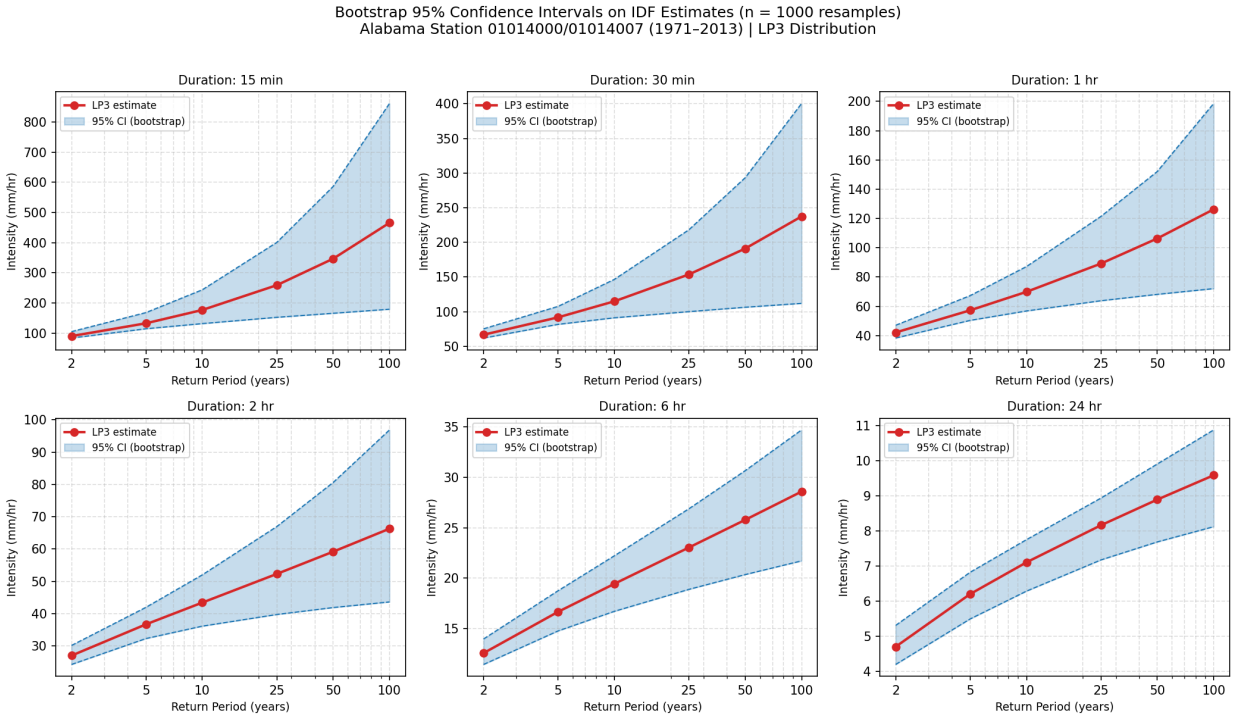

Figure 6: Bootstrap 95% confidence intervals (blue shading) around LP3 point estimates (red solid line) for all six storm durations. Each sub-panel corresponds to one duration. The x-axis is log-scale. Note how the shading (uncertainty band) grows enormously as return period increases, especially at 15 min and 30 min. At 24 hr, the bands remain relatively narrow even at $T = 100$ yr. Based on 1000 bootstrap resamples of the $n = 43$ AMS dataset.

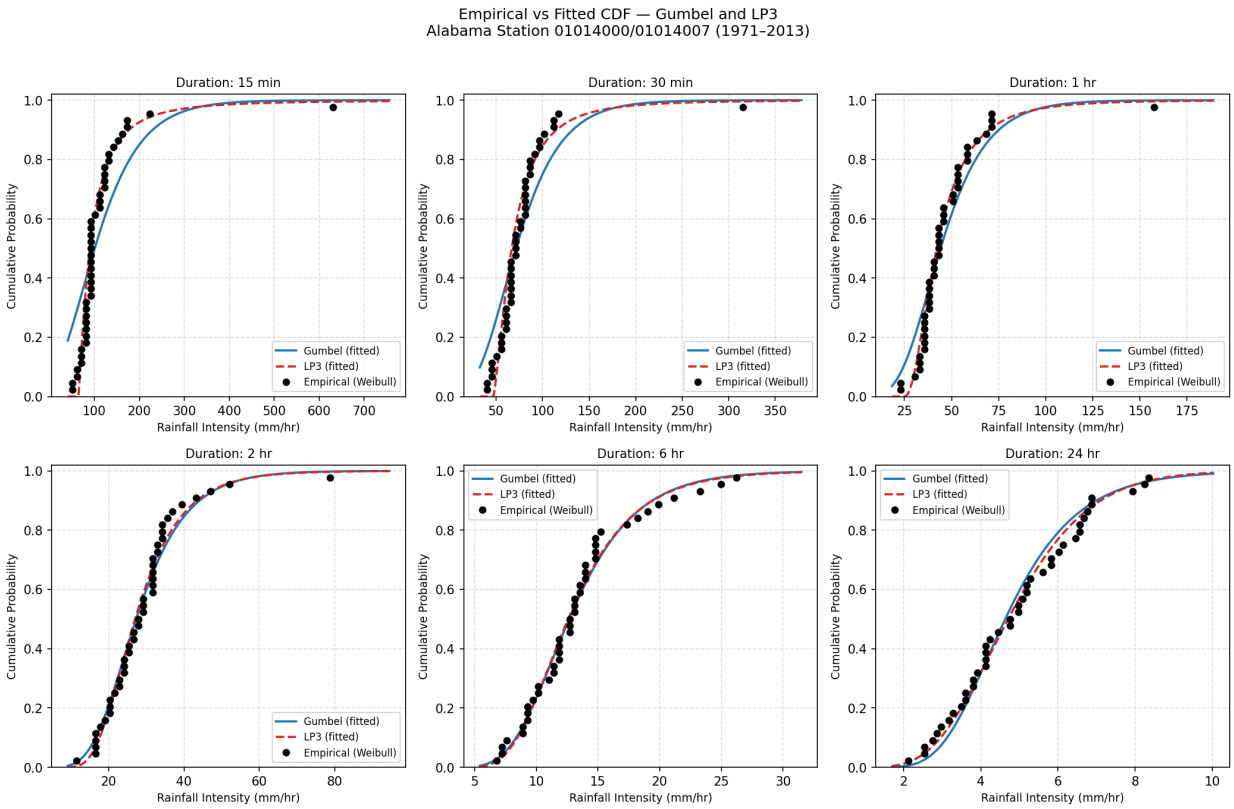

Figure 7: Empirical vs. fitted CDFs for all six storm durations. Black dots = Weibull plotting positions (empirical CDF). Blue line = fitted Gumbel CDF. Red line = fitted LP3 CDF. Both axes: x = rainfall intensity (mm/hr), y = cumulative probability. The KS statistic D_n equals the maximum vertical gap between the dots and a fitted line. At 15 min, the rightmost dot (the 1971 outlier at 629.9 mm/hr) is far from both fitted curves, indicating neither distribution fits that extreme point well. LP3 tracks the body of the data more closely than Gumbel at all durations.

Table 11: Summary of all key numerical results from the full analysis pipeline for NOAA Station 01014000/01014007, Randolph, Alabama (1971–2013).

Finding	Description	Key Number(s)
KS test	Gumbel fails the KS test at 15 min ($D = 0.228 > D_{\text{crit}} = 0.207$). LP3 passes all six durations. LP3 is selected as the best-fit distribution everywhere because its skewness parameter accommodates the heavy upper tail of short-duration AMS data. Gumbel's fixed tail behaviour is insufficient for positively skewed annual maxima driven by rare convective events.	$D_{\text{LP3, 15min}} = 0.181$; $D_{\text{Gumbel, 15min}} = 0.228$; $D_{\text{crit}} = 0.207$
IDF formula	Three-parameter power-law formula $i = aT^b/d^e$ fitted by log-linear ordinary least squares to all 36 LP3 design values (6 durations × 6 return periods). Valid for return periods T = 2–100 yr and durations d = 15 min–24 hr. The exponent b governs return-period amplification; e governs intensity decay with duration.	$a = 36.61$, $b = 0.271$, $e = 0.742$
Formula RMSE	Root mean square error between the fitted formula $i = aT^b/d^e$ and the 36 LP3 point estimates. Most error is concentrated at 15-min, high-T entries where the LP3 skewness creates a very steep tail that the simple power law in T cannot perfectly represent.	RMSE = 22.3 mm/hr
Atlas 14 (low T)	At $T = 2$ yr, our estimates agree with NOAA Atlas 14 Volume 9 within ±4% across all six durations. This excellent agreement validates the LP3 fitting procedure and the full data parsing pipeline for common storm events.	Max deviation at $T = 2$ yr: +6.9% at 24 hr; −3.6% at 1 hr
Atlas 14 (high T)	At $T = 100$ yr, systematic and progressive overestimation develops, largest at short durations. The 15-min, 100-yr intensity is 159% above Atlas 14. This divergence is driven almost entirely by the 1971 outlier distorting the LP3 upper tail. Agreement remains reasonable at 24 hr.	+159% at $T = 100$ yr, 15 min; −8.3% at $T = 100$ yr, 24 hr
Grubbs outlier	The 1971 storm is a statistically significant outlier at the four shortest durations. Its Grubbs statistic at 15 min is nearly twice the critical value. It is NOT a statistical outlier at 6-hr and 24-hr durations, indicating the storm was extremely intense over short windows but produced a normal accumulated depth over longer periods.	$G_{15\text{min}} = 5.87 \gg G_{\text{crit}} = 3.07$; $G_{30\text{min}} = 5.71$; $G_{1\text{hr}} = 5.21$; $G_{6\text{hr}} = 2.79 < 3.07$
Outlier impact	Removing the 1971 event reduces the 100-yr, 15-min design intensity from 465.5 to 187.6 mm/hr — a 60% reduction. The no-outlier estimate (187.6 mm/hr) is within 4.3% of Atlas 14 (179.8 mm/hr), confirming that the outlier is the primary source of Atlas 14 divergence at short durations and high return periods.	−59.7% at $T = 100$ yr, 15 min; −51.7% at $T = 100$ yr, 30 min; +12.8% at $T = 2$ yr, 2 hr
Trend: 15-min	At $T = 100$ yr, Period 1 (1971–1992) gives 691.5 mm/hr vs. 187.6 mm/hr in Period 2 (1993–2013) — largely outlier-driven. Even after outlier removal, short-duration extremes are higher in Period 1, suggesting possible non-stationarity in convective rainfall intensity across the 43-year record.	Factor of 3.7× between periods (with outlier); Factor of ∼ 1.6× (without outlier)
Trend: 24-hr	Period 2 (1993–2013) gives slightly higher 24-hr intensities at all return periods, which is the opposite trend to short durations. This may indicate a shift toward longer-duration, lower-intensity synoptic rainfall events in the more recent climate period.	Period 2 is +6% higher at $T = 100$ yr, 24 hr
Bootstrap CI (short duration)	The 95% bootstrap confidence interval for the 15-min, 100-yr intensity spans 179.2 to 861.0 mm/hr. This enormous range (147% of the point estimate) reflects the fundamental statistical limitation of estimating 100-year events from only 43 data points when one dominant outlier is present in the record.	Width = 681.8 mm/hr; 147% of point estimate (465.5 mm/hr)
Bootstrap CI (long duration)	The 24-hr, 100-yr estimate has a much narrower 95% CI of 8.1–10.9 mm/hr. The lower variability and near-zero skewness ($g = -0.28$) of the 24-hr AMS produce well-constrained LP3 parameters even from only 43 years of data, making long-duration estimates far more reliable.	Width = 2.8 mm/hr; 29% of point estimate (9.6 mm/hr)

16 Broader Implications and Limitations

16.1 Engineering Use of the Derived IDF Curves

For drainage design in the Randolph area using the rational method $Q_p = C \cdot i \cdot A$, the designer can use the empirical formula $i = 36.61\,T^{0.271}/d^{0.742}$. However, given the Atlas 14 comparison results, the following guidance applies:

1. **For $T \leq 5$ yr:** Use our derived values with confidence. They agree within ±14% of Atlas 14, which is within normal engineering uncertainty.
2. **For $T = 10$–25 yr:** Use our values with caution and consider cross-checking against Atlas 14, which is likely more reliable for short-duration rare events.
3. **For $T > 25$ yr at short durations (15 min, 30 min):** *Do not* use our derived values for life-safety or critical infrastructure. The 1971 outlier distorts these estimates severely. Atlas 14 should be the primary reference.
4. **For $T > 25$ yr at long durations (6 hr, 24 hr):** Our estimates are reasonable and agree within about 15% of Atlas 14.

16.2 Limitations of This Analysis

1. Small sample size for extreme events. With 43 years of data, we cannot reliably estimate events with a return period of 100 years. The bootstrap results make this vividly clear. A 100-year event from 43 data points is an extrapolation 2.3 return periods beyond the record length, which is universally acknowledged to have very high uncertainty.

2. Single station. Atlas 14 uses dozens of stations in a regional frequency analysis. This drastically reduces the influence of any single extreme event (like our 1971 storm) and provides much more stable parameter estimates. A single station is appropriate for understanding local behaviour but not for reliable rare-event estimation.

3. Stationarity assumption. The split-period analysis shows that the rainfall regime may have changed between 1971–1992 and 1993–2013. Using the full 43-year record implicitly assumes these changes are negligible, which may not be appropriate for long-range design.

4. Annual Maximum Series vs. Partial Duration Series. The AMS uses only one event per year. The Partial Duration Series (PDS), which Atlas 14 uses, includes all events above a threshold, capturing multiple extreme storms per year when they occur. PDS is generally considered superior for frequent events ($T \leq 5$ yr), where the AMS may underestimate intensity.

5. Measurement resolution. The gauge records in 0.01-inch steps (0.254 mm). For very short durations and small depths, this quantisation error can be significant.

17 Conclusions

This project constructed a complete, from-scratch IDF curve derivation pipeline for NOAA Station 01014000/01014007 in Randolph, Alabama, using 43 years of 15-minute precipitation data. All statistical methods were implemented in Python without any statistical solver library, making every assumption explicit and every calculation reproducible.

The key conclusions are:

1. LP3 is the better distribution. The Kolmogorov–Smirnov test selects Log-Pearson Type III over Gumbel for all six storm durations. Gumbel actually fails the test at the 15-minute duration. LP3's skewness parameter allows it to capture the heavy upper tail of the short-duration AMS caused by rare convective events.

2. The 1971 storm is the dominant data feature. A single extreme storm in August 1971 produced 629.9 mm/hr at the 15-minute duration — nearly 7 times the median annual maximum. The Grubbs test confirms it as a statistically significant outlier at four of six durations. This event inflates the 100-year, 15-minute design intensity from approximately 188 mm/hr (which agrees with Atlas 14) to 465 mm/hr (which is 159% higher than Atlas 14). Engineers using the full-record IDF curves for life-safety design at short durations should be aware of this sensitivity.

3. Uncertainty is very large for rare events. Bootstrap confidence intervals show that the 100-year, 15-minute intensity has a 95% confidence interval spanning from 179 to 861 mm/hr — a range of 682 mm/hr. This is not a rounding error; it reflects the fundamental statistical limitation of estimating 100-year events from 43 data points. Long-duration estimates (24 hr) are much more reliable, with a CI width of only 2.8 mm/hr at $T = 100$ yr.

4. There are signs of non-stationarity. The split-period analysis suggests that short-duration extreme intensities were higher in 1971–1992 than in 1993–2013 (even after removing the 1971 outlier), while long-duration intensities were slightly higher in the later period. This warrants further investigation using formal non-stationary frequency analysis methods.

5. The empirical formula works well. The three-parameter formula $i = 36.61\,T^{0.271}/d^{0.742}$ represents the derived IDF surface with an RMSE of 22.3 mm/hr and is suitable for direct use in stormwater design calculations for return periods up to about 25 years.

References

[1] Perica, S., Martin, D., Pavlovic, S., Roy, I., Laurent, M., Trypaluk, C., Unruh, D., Yekta, M. and Bonnin, G., 2015. *NOAA Atlas 14: Precipitation-Frequency Atlas of the United States, Volume 9 — Southeastern States, Version 2.0.* National Weather Service, NOAA, Silver Spring, MD. Available at: https://hdsc.nws.noaa.gov/pfds/

[2] Koutsoyiannis, D., Kozonis, D. and Manetas, A., 1998. A mathematical framework for studying rainfall intensity-duration-frequency relationships. *Journal of Hydrology*, 206(1–2), pp. 118–135. doi:10.1016/S0022-1694(98)00097-3

[3] Grubbs, F.E., 1950. Sample criteria for testing outlying observations. *The Annals of Mathematical Statistics*, 21(1), pp. 27–58.

[4] Massey, F.J. Jr, 1951. The Kolmogorov–Smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253), pp. 68–78.

[5] Efron, B., 1979. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1), pp. 1–26.

[6] Wilson, E.B. and Hilferty, M.M., 1931. The distribution of chi-square. *Proceedings of the National Academy of Sciences*, 17(12), pp. 684–688.

[7] England, J.F. Jr, Cohn, T.A., Faber, B.A., Stedinger, J.R., Thomas, W.O. Jr, Veilleux, A.G., Kiang, J.E. and Mason, R.R. Jr, 2019. *Guidelines for Determining Flood Flow Frequency — Bulletin 17C*. U.S. Geological Survey Techniques and Methods, Book 4, Chapter B5.

[8] Milly, P.C.D., Betancourt, J., Falkenmark, M., Hirsch, R.M., Kundzewicz, Z.W., Lettenmaier, D.P. and Stouffer, R.J., 2008. Stationarity is dead: whither water management? *Science*, 319(5863), pp. 573–574.

Appendix: Complete Python Source Code

The whole project is available here: GitHub_OjasPathak

The following pages contain the complete, unmodified Python source code for all 13 pipeline scripts. Scripts are presented in execution order (Step 0 through Step 13). The `output/` directory is created automatically; all results are written there. To run the full pipeline from scratch, execute the scripts in order from the project root directory (not from inside the `code/` folder).

Python version: 3.8 or later. **Required packages:** `numpy`, `matplotlib`, `pandas`, `unlzw3` (all installable via `pip`).

Step 0: Extract .tar.Z Archives (`step0_extract.py`)

Purpose: Decompress the raw NOAA DSI-3260 `.tar.Z` files into readable `.dat` files. The `.tar.Z` format uses LZW compression (an old Unix format), handled by the `unlzw3` library. The decompressed TAR archive is then extracted using Python's built-in `tarfile` module.

```python
"""
Step 0: Extract .tar.Z files from Alabama state folder
"""

import os
import tarfile
import unlzw3

# Paths
BASE_DIR         = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
INPUT_FOLDER     = os.path.join(BASE_DIR, "Dataset", "state_data", "01")
OUTPUT_FOLDER    = os.path.join(BASE_DIR, "Dataset", "extracted")

os.makedirs(OUTPUT_FOLDER, exist_ok=True)

# Extract .tar.Z files
files = sorted([f for f in os.listdir(INPUT_FOLDER) if f.endswith(".tar.Z")])

if not files:
    print(f"No .tar.Z files found in: {INPUT_FOLDER}")
    print("Make sure the Dataset/state_data/01/ folder contains the Alabama files.")
else:
    print(f"Found {len(files)} file(s) to extract.\n")
    for filename in files:
        filepath = os.path.join(INPUT_FOLDER, filename)
        print(f"Processing: {filename} ...", end=" ", flush=True)

        # Read and decompress the .Z file using unlzw3 (LZW decompression)
        with open(filepath, "rb") as f:
            compressed = f.read()

        tar_bytes = unlzw3.unlzw(compressed)   # LZW -> raw TAR bytes

        # Write to a temporary .tar file
        temp_tar = os.path.join(OUTPUT_FOLDER, filename.replace(".tar.Z", ".tar"))
        with open(temp_tar, "wb") as f:
            f.write(tar_bytes)

        # Extract the .tar to the output folder
        with tarfile.open(temp_tar) as tar:
            tar.extractall(path=OUTPUT_FOLDER)

        # Remove the temporary .tar file to save space
        os.remove(temp_tar)
        print("Done")

    print(f"\nAll files extracted successfully")
    print(f"Extracted files are in: {OUTPUT_FOLDER}")
```

Listing 1: step0_extract.py — Decompress NOAA .tar.Z archives

Step 1: Parse DSI-3260 Records (`step1_parse.py`)

Purpose: Read every `.dat` file and extract the 15-minute precipitation depth values for our two station IDs. The fixed-width format is parsed by character position. Quality flags are checked; missing and deleted records are skipped. Values are converted from hundredths of an inch to millimetres. The output `clean_data.csv` is used by all subsequent steps.

```python
"""
```

```python
Step 1: Parse all .dat files and output a clean CSV
"""

import os
import pandas as pd

# Paths
BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")
DATA_DIR   = os.path.join(BASE_DIR, "Dataset")

os.makedirs(OUTPUT_DIR, exist_ok=True)

OUTPUT_CSV       = os.path.join(OUTPUT_DIR, "clean_data.csv")
EXTRACTED_FOLDER = os.path.join(DATA_DIR, "extracted")
DAT_FOLDERS      = [
    os.path.join(DATA_DIR, "state_data", "by_month2011"),
    os.path.join(DATA_DIR, "state_data", "by_month2012"),
    os.path.join(DATA_DIR, "state_data", "by_month2013"),
]

# Both IDs refer to the same physical station.
# 01014000 = pre-1993 station code; 01014007 = post-1993 station code
STATION_IDS = {"01014000", "01014007"}


def parse_file(filepath):
    """
    Parse a single DSI-3260 .dat file.
    Returns a list of dicts with keys:
        year, month, day, hour, minute, precip_mm
    """
    records = []

    with open(filepath, "r", errors="replace") as f:
        for line in f:
            line = line.rstrip("\n")

            if len(line) < 30:       # skip short lines
                continue

            # Filter: only 15-minute precipitation records
            record_type  = line[0:3].strip()
            element_type = line[11:15].strip()
            if record_type != "15M" or element_type != "QPCP":
                continue

            # Filter: only our station
            station_id = line[3:11].strip()
            if station_id not in STATION_IDS:
                continue

            # Parse header fields (fixed-width positions)
            units = line[15:17].strip()
            try:
                year  = int(line[17:21])
                month = int(line[21:23])
                day   = int(line[23:27])
                n_val = int(line[27:30])   # number of time/value triplets
            except ValueError:
                continue

            # Parse each time-value-flag triplet (12 characters each)
            pos = 30
            for _ in range(n_val):
                if pos + 12 > len(line):
                    break

                time_str  = line[pos:pos+4]       # HHMM
                value_str = line[pos+4:pos+10].strip()
                flag1     = line[pos+10] if len(line) > pos + 10 else " "
                pos += 12

                # Skip daily total record
                if time_str == "2500":
                    continue
```

```python

                # Skip missing values (099999) and deleted/flagged records
                if value_str == "099999":
                    continue
                if flag1 in ("[", "]", "{", "}", "a", ","):
                    continue

                try:
                    raw_value = int(value_str)
                except ValueError:
                    continue

                # Convert to millimetres
                # HI/HT units = hundredths of an inch
                if units in ("HI", "HT"):
                    precip_mm = raw_value / 100.0 * 25.4
                else:
                    continue

                # Parse hour and minute from HHMM string
                try:
                    hour   = int(time_str[0:2])
                    minute = int(time_str[2:4])
                    if hour == 24:   # midnight represented as 2400
                        hour = 0
                except ValueError:
                    continue

                records.append({
                    "year":      year,
                    "month":     month,
                    "day":       day,
                    "hour":      hour,
                    "minute":    minute,
                    "precip_mm": precip_mm
                })

    return records


# ---- Main ----------------------------------------------------------------
all_records = []

# Scan extracted folder (pre-2011 data from .tar.Z archives)
if os.path.isdir(EXTRACTED_FOLDER):
    for filename in sorted(os.listdir(EXTRACTED_FOLDER)):
        filepath = os.path.join(EXTRACTED_FOLDER, filename)
        if os.path.isfile(filepath):
            print(f"Reading (extracted): {filename}")
            all_records.extend(parse_file(filepath))
else:
    print(f"WARNING: Extracted folder not found: {EXTRACTED_FOLDER}")
    print("Run step0_extract.py first.")

# Scan by_month folders (2011-2013 data in separate .dat files)
for folder in DAT_FOLDERS:
    if not os.path.isdir(folder):
        print(f"Skipping (not found): {folder}")
        continue
    for filename in sorted(os.listdir(folder)):
        if not filename.endswith(".dat"):
            continue
        filepath = os.path.join(folder, filename)
        print(f"Reading (by_month):  {filename}")
        all_records.extend(parse_file(filepath))

# Build DataFrame and save
df = pd.DataFrame(all_records)
df = df.drop_duplicates()
df = df.sort_values(["year", "month", "day", "hour", "minute"])
df.to_csv(OUTPUT_CSV, index=False)

print(f"\nDone")
print(f"Total records: {len(df)}")
print(f"Years in data: {sorted(df['year'].unique())}")
print(f"Saved to: {OUTPUT_CSV}")
```

Listing 2: step1_parse.py — Parse NOAA DSI-3260 fixed-width data files

Step 2: Annual Maximum Series (`step2_ams.py`)

Purpose: Convert the 15-minute depth records into intensity time series (mm/hr) for six storm durations using a rolling-window approach. Extract the annual maximum intensity for each duration and year. Output the 43×6 AMS matrix.

```python
"""
Step 2: Extract Annual Maximum Series (AMS)
"""

import os
import pandas as pd
import numpy as np

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

INPUT_CSV  = os.path.join(OUTPUT_DIR, "clean_data.csv")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "ams.csv")

# Load data and build datetime index
df = pd.read_csv(INPUT_CSV)
df["datetime"] = pd.to_datetime(df[["year", "month", "day", "hour", "minute"]])
df = df.set_index("datetime").sort_index()

# Resample to complete 15-min grid; missing intervals (dry) get 0
ts = df["precip_mm"].resample("15min").sum().fillna(0)

# Duration definitions: label -> number of 15-min steps
durations = {
    "15min": 1,   # 1 x 15 min = 15 min
    "30min": 2,   # 2 x 15 min = 30 min
    "1hr":   4,   # 4 x 15 min =  1 hr
    "2hr":   8,   # 8 x 15 min =  2 hr
    "6hr":   24,  # 24 x 15 min = 6 hr
    "24hr":  96   # 96 x 15 min = 24 hr
}

ams_dict = {}

for label, steps in durations.items():
    print(f"Computing AMS for duration: {label}")

    # Rolling sum of depth over 'steps' consecutive 15-min intervals
    rolling = ts.rolling(window=steps).sum()

    # Convert accumulated depth (mm) to intensity (mm/hr)
    duration_hours = steps * 15 / 60.0       # e.g. 4 steps * 15 min = 1 hr
    intensity      = rolling / duration_hours  # mm / hr

    # Take the single maximum intensity per calendar year
    annual_max = intensity.groupby(intensity.index.year).max()
    ams_dict[label] = annual_max

# Assemble and save
ams_df            = pd.DataFrame(ams_dict)
ams_df.index.name = "year"
ams_df            = ams_df.dropna()   # drop years with incomplete records

ams_df.to_csv(OUTPUT_CSV)

print(f"\nAMS saved to: {OUTPUT_CSV}")
print(f"Years in AMS: {ams_df.index.min()} -- {ams_df.index.max()} "
      f"({len(ams_df)} years)")
print("\nDescriptive statistics (intensity in mm/hr):")
print(ams_df.describe().round(3))
```

Listing 3: step2_ams.py — Extract Annual Maximum Series using rolling windows

Step 3: Distribution Fitting (step3_distributions.py)

Purpose: Fit Gumbel (EV-I) and Log-Pearson Type III distributions to the AMS for each duration using the Method of Moments. Save parameters for use in subsequent steps.

```python
"""
Step 3: Fit Gumbel and LP3 frequency distributions to the AMS
"""

import os
import pandas as pd
import numpy as np

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

INPUT_CSV  = os.path.join(OUTPUT_DIR, "ams.csv")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "params.csv")


def gumbel_params(data):
    """
    Estimate Gumbel (EV-I) parameters using Method of Moments.
    - sigma (scale)    = (sqrt(6) / pi) * s
    - mu (location)    = x_bar - 0.5772 * sigma
    where x_bar = sample mean, s = sample std dev (ddof=1).
    """
    xbar  = np.mean(data)
    s     = np.std(data, ddof=1)
    sigma = (np.sqrt(6) / np.pi) * s
    mu    = xbar - 0.5772 * sigma
    return mu, sigma


def lp3_params(data):
    """
    Estimate LP3 parameters using Method of Moments on log10-transformed data.
    - ybar : mean of log10(data)
    - sy   : standard deviation of log10(data), ddof=1
    - g    : skewness coefficient of log10(data)
    """
    y    = np.log10(data)
    n    = len(y)
    ybar = np.mean(y)
    sy   = np.std(y, ddof=1)
    g    = (n * np.sum((y - ybar)**3)) / ((n - 1) * (n - 2) * sy**3)
    return ybar, sy, g


# Main
ams       = pd.read_csv(INPUT_CSV, index_col="year")
durations = ams.columns.tolist()
results   = []

for dur in durations:
    data = ams[dur].dropna().values

    mu, sigma      = gumbel_params(data)
    ybar, sy, g    = lp3_params(data)

    results.append({
        "duration":     dur,
        "n":            len(data),
        "gumbel_mu":    round(mu,    4),
        "gumbel_sigma": round(sigma, 4),
        "lp3_ybar":     round(ybar,  4),
        "lp3_sy":       round(sy,    4),
        "lp3_g":        round(g,     4)
    })

    print(f"{dur}:")
    print(f"  Gumbel -> mu = {mu:.3f},   sigma = {sigma:.3f}")
    print(f"  LP3    -> ybar = {ybar:.3f}, sy = {sy:.3f}, g = {g:.3f}")

params_df = pd.DataFrame(results)
params_df.to_csv(OUTPUT_CSV, index=False)
print(f"\nParameters saved to: {OUTPUT_CSV}")
```

Listing 4: step3_distributions.py — Fit Gumbel and LP3 distributions by Method of Moments

Step 4: KS Goodness-of-Fit Test (`step4_ks_test.py`)

Purpose: Apply the two-sided Kolmogorov–Smirnov test to both Gumbel and LP3 fits. Select the distribution with the smaller KS statistic that also passes the test. Since no `scipy` is used, the LP3 CDF is evaluated by building a fine grid of quantiles and interpolating; the standard normal inverse CDF is computed via a rational approximation.

```python
"""
Step 4: Kolmogorov-Smirnov Goodness-of-Fit Test
"""

import os
import pandas as pd
import numpy as np

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

AMS_CSV    = os.path.join(OUTPUT_DIR, "ams.csv")
PARAMS_CSV = os.path.join(OUTPUT_DIR, "params.csv")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "ks_results.csv")


def gumbel_cdf(x, mu, sigma):
    """Gumbel CDF: F(x) = exp(-exp(-(x - mu) / sigma))"""
    return np.exp(-np.exp(-(x - mu) / sigma))


def inv_normal_cdf(p):
    """
    Rational approximation for the inverse standard normal CDF.
    Abramowitz and Stegun (1964), accurate to ~4 decimal places.
    """
    if p <= 0.0: return -8.0
    if p >= 1.0: return  8.0
    if p < 0.5:  return -inv_normal_cdf(1.0 - p)
    t = np.sqrt(-2.0 * np.log(1.0 - p))
    c = [2.515517, 0.802853, 0.010328]
    d = [1.432788, 0.189269, 0.001308]
    return t - (c[0] + c[1]*t + c[2]*t**2) / \
               (1.0 + d[0]*t + d[1]*t**2 + d[2]*t**3)


def kt_wilson_hilferty(g, T):
    """
    Wilson-Hilferty (1931) frequency factor K_T for Pearson Type III.
    K_T = (2/g) * [(1 + g*z/6 - g^2/36)^3 - 1]
    where z = standard normal quantile at non-exceedance prob p = 1 - 1/T.
    """
    p = 1.0 - 1.0 / T
    z = inv_normal_cdf(p)
    if abs(g) < 1e-6:
        return z   # reduces to normal (log-normal) when g = 0
    return (2.0 / g) * ((1.0 + g*z/6.0 - g**2/36.0)**3 - 1.0)


def lp3_cdf(x_vals, ybar, sy, g):
    """
    Compute LP3 CDF at each value in x_vals.
    Method: build a fine probability grid, compute quantiles via
    Wilson-Hilferty, interpolate the inverse to get F(x).
    """
    p_grid = np.linspace(0.0001, 0.9999, 50000)
    z_grid = np.array([inv_normal_cdf(p) for p in p_grid])

    if abs(g) < 1e-6:
        kt_grid = z_grid
    else:
        kt_grid = (2.0/g) * ((1.0 + g*z_grid/6.0 - g**2/36.0)**3 - 1.0)

    x_grid   = 10.0 ** (ybar + kt_grid * sy)
    sort_idx = np.argsort(x_grid)
    x_sorted = x_grid[sort_idx]
```

```python
    p_sorted = p_grid[sort_idx]

    return np.interp(x_vals, x_sorted, p_sorted)   # interpolate CDF


def ks_statistic(data_sorted, cdf_values):
    """
    Two-sided KS statistic.
    D = max |F_empirical(x_i) - F_theoretical(x_i)|
    Empirical CDF uses i/n (end-point of Weibull plotting position step).
    Critical value at alpha = 0.05: D_crit = 1.36 / sqrt(n).
    """
    n       = len(data_sorted)
    emp_cdf = np.arange(1, n+1) / n     # empirical CDF
    D       = np.max(np.abs(emp_cdf - cdf_values))
    D_crit  = 1.36 / np.sqrt(n)
    return D, D_crit, bool(D < D_crit)


# Main
ams    = pd.read_csv(AMS_CSV,    index_col="year")
params = pd.read_csv(PARAMS_CSV)
results = []

for _, row in params.iterrows():
    dur  = row["duration"]
    data = np.sort(ams[dur].dropna().values)

    # Gumbel KS test
    g_cdf_vals      = gumbel_cdf(data, row["gumbel_mu"], row["gumbel_sigma"])
    D_g, Dc, pass_g = ks_statistic(data, g_cdf_vals)

    # LP3 KS test
    l_cdf_vals      = lp3_cdf(data, row["lp3_ybar"], row["lp3_sy"], row["lp3_g"])
    D_l, Dc, pass_l = ks_statistic(data, l_cdf_vals)

    best = "LP3" if D_l <= D_g else "Gumbel"

    print(f"{dur}: "
          f"Gumbel D = {D_g:.4f} ({'PASS' if pass_g else 'FAIL'}), "
          f"LP3 D = {D_l:.4f} ({'PASS' if pass_l else 'FAIL'}), "
          f"D_crit = {Dc:.4f} | Best fit: {best}")

    results.append({
        "duration":      dur,
        "D_gumbel":      round(D_g, 4),
        "D_lp3":         round(D_l, 4),
        "D_critical":    round(Dc,  4),
        "gumbel_passes": pass_g,
        "lp3_passes":    pass_l,
        "best_fit":      best
    })

ks_df = pd.DataFrame(results)
ks_df.to_csv(OUTPUT_CSV, index=False)
print(f"\nKS results saved to: {OUTPUT_CSV}")
```

Listing 5: step4_ks_test.py — Two-sided Kolmogorov–Smirnov goodness-of-fit test

Step 5: IDF Table (step5_return_periods.py)

Purpose: Using the best-fit distribution from Step 4 (LP3 for all durations), compute the design rainfall intensity at return periods $T \in \{2, 5, 10, 25, 50, 100\}$ years for all six durations. This produces the primary engineering deliverable: the IDF table.

```python
"""
Step 5: Compute Return Period Rainfall Intensities
"""

import os
import pandas as pd
import numpy as np

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

```

```python
PARAMS_CSV = os.path.join(OUTPUT_DIR, "params.csv")
KS_CSV     = os.path.join(OUTPUT_DIR, "ks_results.csv")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "idf_table.csv")

RETURN_PERIODS = [2, 5, 10, 25, 50, 100]


def gumbel_quantile(T, mu, sigma):
    """Gumbel T-year quantile: x_T = mu - sigma * ln(-ln(1 - 1/T))"""
    return mu - sigma * np.log(-np.log(1.0 - 1.0 / T))


def inv_normal_cdf(p):
    """Rational approximation for inverse standard normal CDF."""
    if p <= 0.0: return -8.0
    if p >= 1.0: return  8.0
    if p < 0.5:  return -inv_normal_cdf(1.0 - p)
    t = np.sqrt(-2.0 * np.log(1.0 - p))
    c = [2.515517, 0.802853, 0.010328]
    d = [1.432788, 0.189269, 0.001308]
    return t - (c[0] + c[1]*t + c[2]*t**2) / \
               (1.0 + d[0]*t + d[1]*t**2 + d[2]*t**3)


def kt_wilson_hilferty(g, T):
    """Wilson-Hilferty frequency factor for LP3."""
    p = 1.0 - 1.0 / T
    z = inv_normal_cdf(p)
    if abs(g) < 1e-6:
        return z
    return (2.0 / g) * ((1.0 + g*z/6.0 - g**2/36.0)**3 - 1.0)


def lp3_quantile(T, ybar, sy, g):
    """LP3 T-year quantile: x_T = 10^(ybar + K_T * sy)"""
    return 10.0 ** (ybar + kt_wilson_hilferty(g, T) * sy)


# Main
params = pd.read_csv(PARAMS_CSV)
ks     = pd.read_csv(KS_CSV)
rows   = []

for _, p_row in params.iterrows():
    dur      = p_row["duration"]
    best_fit = ks[ks["duration"] == dur]["best_fit"].values[0]

    for T in RETURN_PERIODS:
        if best_fit == "Gumbel":
            intensity = gumbel_quantile(T, p_row["gumbel_mu"],
                                           p_row["gumbel_sigma"])
        else:
            intensity = lp3_quantile(T, p_row["lp3_ybar"],
                                        p_row["lp3_sy"],
                                        p_row["lp3_g"])
        rows.append({
            "duration":            dur,
            "T_years":             T,
            "best_fit_dist":       best_fit,
            "intensity_mm_per_hr": round(intensity, 3)
        })

idf_df = pd.DataFrame(rows)
idf_df.to_csv(OUTPUT_CSV, index=False)

print("IDF Table (intensity in mm/hr):")
pivot = idf_df.pivot(index="duration", columns="T_years",
                     values="intensity_mm_per_hr")
print(pivot.to_string())
print(f"\nIDF table saved to: {OUTPUT_CSV}")
```

Listing 6: step5_return_periods.py — Compute IDF intensities for all return periods

Step 6: Empirical IDF Formula (`step6_idf_formula.py`)

Purpose: Fit the three-parameter power-law formula $i = a\,T^b/d^e$ to the 36 IDF data points using log-linear ordinary least squares. The normal equations are solved with `numpy.linalg.solve`.

```python
"""
Step 6: Fit Empirical IDF Formula using Linear Least Squares
"""

import os
import pandas as pd
import numpy as np

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

INPUT_CSV  = os.path.join(OUTPUT_DIR, "idf_table.csv")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "idf_formula_params.csv")

# Duration in hours (for the d variable in the formula)
DUR_MAP = {
    "15min": 0.25,
    "30min": 0.50,
    "1hr":   1.00,
    "2hr":   2.00,
    "6hr":   6.00,
    "24hr":  24.0
}

idf = pd.read_csv(INPUT_CSV)
idf["d_hours"] = idf["duration"].map(DUR_MAP)

# Log-linearise: ln(i) = ln(a) + b*ln(T) - e*ln(d)
# Build design matrix A with rows [1, ln(T), -ln(d)]
ln_i = np.log(idf["intensity_mm_per_hr"].values)
ln_T = np.log(idf["T_years"].values)
ln_d = np.log(idf["d_hours"].values)

A = np.column_stack([
    np.ones(len(ln_i)),   # coefficient for ln(a)
    ln_T,                 # coefficient for b
    -ln_d                 # coefficient for -e (note the negative sign)
])
y = ln_i

# OLS solution: beta = (A^T A)^{-1} A^T y
# Solved via linalg.solve (more stable than explicit matrix inverse)
ATA  = A.T @ A
ATy  = A.T @ y
beta = np.linalg.solve(ATA, ATy)

ln_a, b, e = beta
a = float(np.exp(ln_a))
b = float(b)
e = float(e)

print(f"Fitted IDF Formula: i = {a:.4f} * T^{b:.4f} / d^{e:.4f}")

# Residuals and RMSE (back in original units mm/hr)
i_pred    = a * idf["T_years"].values**b / idf["d_hours"].values**e
residuals = idf["intensity_mm_per_hr"].values - i_pred
rmse      = float(np.sqrt(np.mean(residuals**2)))
print(f"RMSE: {rmse:.4f} mm/hr")

# Save formula parameters
result = pd.DataFrame([{
    "a":    round(a,    4),
    "b":    round(b,    4),
    "e":    round(e,    4),
    "RMSE": round(rmse, 4)
}])
result.to_csv(OUTPUT_CSV, index=False)
print(f"Formula parameters saved to: {OUTPUT_CSV}")
```

Listing 7: step6_idf_formula.py — Fit i = a*T$^b/d^e bylog - linearleastsquares$

Step 7: IDF Curve Plot (step7_plot.py)

Purpose: Generate the IDF curve family plot (Figure 1) on a log–log scale. Scatter points show the LP3 design values; solid curves show the fitted empirical formula $i = 36.61\,T^{0.271}/d^{0.742}$ evaluated on a fine duration grid.

```python
"""
Step 7: Plot the Final IDF Curves
"""

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

IDF_CSV    = os.path.join(OUTPUT_DIR, "idf_table.csv")
PARAMS_CSV = os.path.join(OUTPUT_DIR, "idf_formula_params.csv")
OUTPUT_PNG = os.path.join(OUTPUT_DIR, "idf_curves.png")

DUR_MAP   = {"15min": 0.25, "30min": 0.5, "1hr": 1.0,
             "2hr": 2.0,   "6hr": 6.0,   "24hr": 24.0}
DUR_ORDER = ["15min", "30min", "1hr", "2hr", "6hr", "24hr"]
T_VALUES  = [2, 5, 10, 25, 50, 100]
COLORS    = ["#1f77b4", "#ff7f0e", "#2ca02c",
             "#d62728", "#9467bd", "#8c564b"]

idf    = pd.read_csv(IDF_CSV)
params = pd.read_csv(PARAMS_CSV).iloc[0]
a, b, e = params["a"], params["b"], params["e"]

# Fine duration grid for smooth curves (log-spaced)
d_fine = np.logspace(np.log10(0.25), np.log10(24), 200)

fig, ax = plt.subplots(figsize=(10, 7))

for T, color in zip(T_VALUES, COLORS):
    # Scatter points from LP3 table
    x_pts = [DUR_MAP[d] for d in DUR_ORDER]
    y_pts = []
    for d in DUR_ORDER:
        val = idf[(idf["duration"] == d) &
                  (idf["T_years"] == T)]["intensity_mm_per_hr"].values
        y_pts.append(val[0] if len(val) > 0 else np.nan)
    ax.plot(x_pts, y_pts, "o", color=color, markersize=6, zorder=3)

    # Smooth fitted formula curve
    i_fine = a * T**b / d_fine**e
    ax.plot(d_fine, i_fine, "-", color=color,
            linewidth=1.8, label=f"T = {T} years")

ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("Duration", fontsize=12)
ax.set_ylabel("Rainfall Intensity (mm/hr)", fontsize=12)
ax.set_title(
    "IDF Curves -- Alabama Station 01014000/01014007\n"
    f"NOAA DSI-3260 (1971-2013) | "
    f"i = {a:.2f} x T^{{{b:.3f}}} / d^{{{e:.3f}}}",
    fontsize=12)
ax.legend(title="Return Period", fontsize=10)
ax.grid(True, which="both", linestyle="--", alpha=0.5)
ax.set_xticks([0.25, 0.5, 1.0, 2.0, 6.0, 24.0])
ax.set_xticklabels(["15 min", "30 min", "1 hr",
                    "2 hr",   "6 hr",   "24 hr"],
                   rotation=30, ha="right")

plt.tight_layout()
plt.savefig(OUTPUT_PNG, dpi=150)
print(f"Plot saved to: {OUTPUT_PNG}")
```

Listing 8: step7_plot.py — Plot the IDF curve family on a log-log scale

Step 8: Atlas 14 Comparison (step8_atlas14_compare.py)

Purpose: Hard-code the NOAA Atlas 14 Volume 9 precipitation depths for our station (retrieved March 2026 from https://hdsc.nws.noaa.gov/pfds/), convert to intensities in mm/hr, merge with our derived IDF table, compute percentage differences, and produce Figure 2.

```python
"""
Step 8: Compare Derived IDF Values Against NOAA Atlas 14
"""

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import matplotlib.patches as mpatches

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

IDF_CSV    = os.path.join(OUTPUT_DIR, "idf_table.csv")
OUTPUT_PNG = os.path.join(OUTPUT_DIR, "atlas14_comparison.png")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "atlas14_comparison.csv")

INCH_TO_MM = 25.4

# NOAA Atlas 14, Vol. 9, Ver. 2
# Station: Randolph, AL (32.8879 N, 86.8772 W)
# Depths in inches per duration (ARI = annual recurrence interval = return period)
atlas14_depth_inches = {
    "15min": {2: 0.911, 5: 1.10,  10: 1.25,  25: 1.46,  50: 1.61,  100: 1.77},
    "30min": {2: 1.32,  5: 1.60,  10: 1.82,  25: 2.12,  50: 2.58,  100: 2.81},
    "1hr":   {2: 1.72,  5: 2.07,  10: 2.36,  25: 2.78,  50: 3.10,  100: 3.44},
    "2hr":   {2: 2.12,  5: 2.54,  10: 2.90,  25: 3.43,  50: 3.85,  100: 4.30},
    "6hr":   {2: 2.88,  5: 3.46,  10: 4.00,  25: 4.83,  50: 5.54,  100: 6.31},
    "24hr":  {2: 4.16,  5: 5.13,  10: 6.03,  25: 7.41,  50: 8.60,  100: 9.88},
}

DUR_HOURS = {"15min": 0.25, "30min": 0.50, "1hr": 1.00,
             "2hr": 2.00,   "6hr": 6.00,   "24hr": 24.0}
T_VALS = [2, 5, 10, 25, 50, 100]

# Convert Atlas 14 depths to intensities
atlas_rows = []
for dur, t_dict in atlas14_depth_inches.items():
    for T, depth_in in t_dict.items():
        intensity_mm_hr = (depth_in / DUR_HOURS[dur]) * INCH_TO_MM
        atlas_rows.append({"duration": dur, "T_years": T,
                           "atlas14_intensity": round(intensity_mm_hr, 3)})
atlas_df = pd.DataFrame(atlas_rows)

# Load our computed IDF and merge
our_idf = pd.read_csv(IDF_CSV)
merged  = pd.merge(our_idf, atlas_df, on=["duration", "T_years"])
merged["diff_mm_hr"]      = round(merged["intensity_mm_per_hr"]
                                  - merged["atlas14_intensity"], 3)
merged["pct_difference"] = round((merged["diff_mm_hr"]
                                  / merged["atlas14_intensity"]) * 100, 1)
merged.to_csv(OUTPUT_CSV, index=False)
print(f"Mean absolute % difference: "
      f"{merged['pct_difference'].abs().mean():.1f}%")
```

Listing 9: step8_atlas14_compare.py — Compare derived IDF against NOAA Atlas 14

Step 11: Grubbs Outlier Detection (step11_outlier_detection.py)

Purpose: Apply the two-sided Grubbs test to each AMS column. Compute the t-distribution critical value using the Cornish–Fisher expansion (no `scipy`). Refit LP3 with and without the confirmed outlier and quantify the percentage impact on IDF estimates.

```python
"""
Step 11: Outlier Detection using Grubbs Test
"""

import os
import pandas as pd
```

```python
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")

AMS_CSV        = os.path.join(OUTPUT_DIR, "ams.csv")
OUTPUT_PNG     = os.path.join(OUTPUT_DIR, "outlier_analysis.png")
OUTPUT_CSV     = os.path.join(OUTPUT_DIR, "outlier_results.csv")
OUTPUT_IDF_CSV = os.path.join(OUTPUT_DIR, "idf_table_no_outlier.csv")

RETURN_PERIODS = [2, 5, 10, 25, 50, 100]


def inv_normal_cdf(p):
    """Rational approximation for inverse standard normal CDF."""
    if p <= 0.0: return -8.0
    if p >= 1.0: return  8.0
    if p < 0.5:  return -inv_normal_cdf(1.0 - p)
    t = np.sqrt(-2.0 * np.log(1.0 - p))
    c = [2.515517, 0.802853, 0.010328]
    d = [1.432788, 0.189269, 0.001308]
    return t - (c[0] + c[1]*t + c[2]*t**2) / \
               (1.0 + d[0]*t + d[1]*t**2 + d[2]*t**3)


def t_inv_approx(p, df):
    """
    Approximate inverse t-distribution CDF using Cornish-Fisher expansion.
    Used for the Grubbs critical value calculation.
    """
    z = inv_normal_cdf(p)
    t_approx = z + (z**3 + z) / (4*df) + \
               (5*z**5 + 16*z**3 + 3*z) / (96*df**2)
    return t_approx


def grubbs_test(data, alpha=0.05):
    """
    Two-sided Grubbs test for a single outlier.
    G_stat = max|xi - xbar| / s
    G_crit = ((n-1)/sqrt(n)) * sqrt(t^2 / (n-2+t^2))
    where t is the t-distribution critical value at alpha/(2n), df=n-2.
    Returns: G_stat, G_crit, outlier_index, is_outlier
    """
    n    = len(data)
    xbar = np.mean(data)
    s    = np.std(data, ddof=1)

    deviations  = np.abs(data - xbar)
    G_stat      = np.max(deviations) / s
    outlier_idx = int(np.argmax(deviations))

    p_t    = alpha / (2 * n)              # tail probability for Bonferroni
    t_crit = t_inv_approx(1.0 - p_t, df=n - 2)
    G_crit = ((n - 1) / np.sqrt(n)) * \
             np.sqrt(t_crit**2 / (n - 2 + t_crit**2))

    return G_stat, G_crit, outlier_idx, bool(G_stat > G_crit)


def lp3_params(data):
    """LP3 parameters from log10-transformed data, Method of Moments."""
    y    = np.log10(data)
    ybar = np.mean(y)
    sy   = np.std(y, ddof=1)
    n    = len(y)
    g    = (n * np.sum((y - ybar)**3)) / ((n - 1) * (n - 2) * sy**3)
    return ybar, sy, g


def kt_wilson_hilferty(g, T):
    """Wilson-Hilferty K_T for LP3."""
    p = 1.0 - 1.0 / T
    z = inv_normal_cdf(p)
    if abs(g) < 1e-6:
```

```
        return z
    return (2.0 / g) * ((1.0 + g*z/6.0 - g**2/36.0)**3 - 1.0)


def lp3_quantile(T, ybar, sy, g):
    """LP3 T-year quantile."""
    return 10.0 ** (ybar + kt_wilson_hilferty(g, T) * sy)


# Main
ams      = pd.read_csv(AMS_CSV, index_col="year")
out_rows = []
idf_rows = []

for dur in ams.columns:
    data  = ams[dur].dropna()
    years = data.index.values
    vals  = data.values

    G_stat, G_crit, idx, is_outlier = grubbs_test(vals)

    out_year = int(years[idx]) if is_outlier else None
    out_val  = float(vals[idx]) if is_outlier else None

    print(f"{dur}: G={G_stat:.4f}, G_crit={G_crit:.4f}, "
          f"Outlier={'YES (year '+str(out_year)+')' if is_outlier else 'no'}")

    out_rows.append({
        "duration":     dur,
        "G_stat":       round(G_stat, 4),
        "G_crit":       round(G_crit, 4),
        "is_outlier":   is_outlier,
        "outlier_year": out_year,
        "outlier_value":round(out_val, 3) if out_val else None
    })

    # Refit LP3 with and without the outlier
    ybar_full, sy_full, g_full = lp3_params(vals)
    clean = np.delete(vals, idx) if is_outlier else vals
    ybar_c, sy_c, g_c = lp3_params(clean)

    for T in RETURN_PERIODS:
        i_full  = lp3_quantile(T, ybar_full, sy_full, g_full)
        i_clean = lp3_quantile(T, ybar_c,    sy_c,    g_c)
        pct     = (i_clean - i_full) / i_full * 100
        idf_rows.append({
            "duration":                  dur,
            "T_years":                   T,
            "intensity_with_outlier":    round(i_full,  3),
            "intensity_without_outlier": round(i_clean, 3),
            "pct_change":                round(pct, 2)
        })

pd.DataFrame(out_rows).to_csv(OUTPUT_CSV, index=False)
pd.DataFrame(idf_rows).to_csv(OUTPUT_IDF_CSV, index=False)
print(f"Outlier results saved to: {OUTPUT_CSV}")
```

Listing 10: step11_outlier_detection.py — Grubbs test and outlier impact analysis

Step 12: Bootstrap Confidence Intervals (`step12_bootstrap_ci.py`)

Purpose: Quantify sampling uncertainty in LP3 IDF estimates using the non-parametric bootstrap. For each duration, 1000 resamples (with replacement) of the AMS are drawn; LP3 is refitted to each; quantiles are computed. The 2.5th and 97.5th percentiles form the 95% CI.

```
"""
Step 12: Bootstrap 95% Confidence Intervals on IDF Estimates
"""

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_DIR = os.path.join(BASE_DIR, "output")
```

```python

AMS_CSV    = os.path.join(OUTPUT_DIR, "ams.csv")
OUTPUT_PNG = os.path.join(OUTPUT_DIR, "bootstrap_ci.png")
OUTPUT_CSV = os.path.join(OUTPUT_DIR, "bootstrap_ci.csv")

RETURN_PERIODS = [2, 5, 10, 25, 50, 100]
N_BOOTSTRAP    = 1000
SEED           = 42
np.random.seed(SEED)   # ensures reproducibility


def lp3_params(data):
    """LP3 parameters, Method of Moments on log10 data."""
    y    = np.log10(data)
    ybar = np.mean(y)
    sy   = np.std(y, ddof=1)
    n    = len(y)
    g    = (n * np.sum((y - ybar)**3)) / ((n - 1) * (n - 2) * sy**3)
    return ybar, sy, g


def inv_normal_cdf(p):
    """Rational approximation for inverse standard normal CDF."""
    if p <= 0.0: return -8.0
    if p >= 1.0: return  8.0
    if p < 0.5:  return -inv_normal_cdf(1.0 - p)
    t = np.sqrt(-2.0 * np.log(1.0 - p))
    c = [2.515517, 0.802853, 0.010328]
    d = [1.432788, 0.189269, 0.001308]
    return t - (c[0] + c[1]*t + c[2]*t**2) / \
               (1.0 + d[0]*t + d[1]*t**2 + d[2]*t**3)


def kt_wilson_hilferty(g, T):
    """Wilson-Hilferty K_T for LP3."""
    p = 1.0 - 1.0 / T
    z = inv_normal_cdf(p)
    if abs(g) < 1e-6:
        return z
    return (2.0 / g) * ((1.0 + g*z/6.0 - g**2/36.0)**3 - 1.0)


def lp3_quantile(T, ybar, sy, g):
    """LP3 T-year quantile."""
    return 10.0 ** (ybar + kt_wilson_hilferty(g, T) * sy)


# Main
ams       = pd.read_csv(AMS_CSV, index_col="year")
DUR_ORDER = ["15min", "30min", "1hr", "2hr", "6hr", "24hr"]
all_rows  = []
results   = {}

print(f"Running {N_BOOTSTRAP} bootstrap resamples per duration...\n")

for dur in DUR_ORDER:
    data = ams[dur].dropna().values
    n    = len(data)
    results[dur] = {T: np.zeros(N_BOOTSTRAP) for T in RETURN_PERIODS}

    for b_iter in range(N_BOOTSTRAP):
        # Resample with replacement
        sample      = data[np.random.randint(0, n, size=n)]
        ybar, sy, g = lp3_params(sample)
        for T in RETURN_PERIODS:
            results[dur][T][b_iter] = lp3_quantile(T, ybar, sy, g)

    # Point estimates from the original data
    ybar0, sy0, g0 = lp3_params(data)

    for T in RETURN_PERIODS:
        boot_vals = results[dur][T]
        point_est = lp3_quantile(T, ybar0, sy0, g0)
        ci_lo     = float(np.percentile(boot_vals, 2.5))
        ci_hi     = float(np.percentile(boot_vals, 97.5))
        all_rows.append({
```

```
            "duration":   dur,
            "T_years":    T,
            "point_est":  round(point_est, 3),
            "ci_lo_2p5":  round(ci_lo,     3),
            "ci_hi_97p5": round(ci_hi,     3),
            "ci_width":   round(ci_hi - ci_lo, 3)
        })

    print(f"  {dur}: done.")

ci_df = pd.DataFrame(all_rows)
ci_df.to_csv(OUTPUT_CSV, index=False)
print(f"\nCSV saved to: {OUTPUT_CSV}")
```

Listing 11: step12_bootstrap_ci.py — Bootstrap 95% confidence intervals for IDF estimates

---

End of Appendix. All scripts produce outputs saved to the `output/` *directory. Run them in order from Step 0 to Step 13 from the project root directory. Steps 0 and 1 need to be run only once; Steps 2–13 can be re-run independently once* `clean_data.csv` *and* `ams.csv` *exist.*